PicoGL

Release 1.0.0

PicoGL Contributors

Sep 06, 2025

USER GUIDE

PicoGL is a lightweight Python OpenGL wrapper designed for educational purposes and simple 3D graphics applications. It provides a clean, easy-to-use interface for OpenGL operations while maintaining compatibility with both modern and legacy OpenGL systems.

CHAPTER

ONE

FEATURES

- **Cross-platform compatibility**: Works on Windows, macOS, and Linux
- **Legacy OpenGL support**: Compatible with OpenGL 1.x, 2.x, and 3.3+
- **Educational focus**: Clean, well-documented code for learning OpenGL
- **Multiple backends**: Support for both modern and legacy rendering pipelines
- **Comprehensive examples**: From simple cubes to complex molecular visualizations
- **Unit testing**: Full test coverage for all major components

CHAPTER

TWO

QUICK START

```
from picogl.renderer import MeshData
from picogl.ui.backend.glut.window.object import RenderWindow

# Create mesh data
data = MeshData.from_raw(vertices=vertices, colors=colors)

# Create render window
window = RenderWindow(width=800, height=600, data=data)
window.initialize()
window.run()
```

CHAPTER THREE

INSTALLATION

```
# Install from PyPI (when available)
pip install picogl

# Or install from source
git clone https://github.com/your-org/picogl.git
cd picogl
pip install -e .
```

CHAPTER FOUR

REQUIREMENTS

- Python 3.7+
- PyOpenGL
- NumPy
- PyGLM (for matrix operations)
- Pillow (for texture loading)

CHAPTER

FIVE

OPTIONAL DEPENDENCIES

- PyQt5/PyQt6 (for Qt-based UI backends)
- GLUT (for GLUT-based examples)

CHAPTER

SIX

CONTENTS

6.1 Installation

PicoGL can be installed from source or from PyPI (when available). This guide covers installation on different platforms and troubleshooting common issues.

6.1.1 Prerequisites

Before installing PicoGL, ensure you have the following:

- **Python 3.7 or higher**
- **OpenGL drivers** for your graphics card
- **Development tools** (for building from source)

6.1.2 Platform-Specific Requirements

Windows

1. **Install Python 3.7+** from python.org
2. **Install Visual Studio Build Tools** (for compiling extensions)
3. **Update graphics drivers** to the latest version

```
# Install PicoGL
pip install picogl

# Or install with all dependencies
pip install picogl[all]
```

macOS

1. **Install Python 3.7+** using Homebrew or from python.org
2. **Install Xcode Command Line Tools**:

   ```
   xcode-select --install
   ```

3. **Install XQuartz** (for X11 support):

   ```
   brew install --cask xquartz
   ```

```
# Install PicoGL
pip install picogl

# Or install with all dependencies
pip install picogl[all]
```

Linux (Ubuntu/Debian)

1. **Install Python 3.7+** and development tools:

```
sudo apt update
sudo apt install python3 python3-pip python3-dev
sudo apt install build-essential libgl1-mesa-dev
```

2. **Install OpenGL libraries**:

```
sudo apt install libgl1-mesa-glx libglu1-mesa
```

```
# Install PicoGL
pip install picogl

# Or install with all dependencies
pip install picogl[all]
```

Linux (CentOS/RHEL/Fedora)

1. **Install Python 3.7+** and development tools:

```
# CentOS/RHEL
sudo yum install python3 python3-pip python3-devel
sudo yum groupinstall "Development Tools"

# Fedora
sudo dnf install python3 python3-pip python3-devel
sudo dnf groupinstall "Development Tools"
```

2. **Install OpenGL libraries**:

```
# CentOS/RHEL
sudo yum install mesa-libGL mesa-libGLU

# Fedora
sudo dnf install mesa-libGL mesa-libGLU
```

```
# Install PicoGL
pip install picogl

# Or install with all dependencies
pip install picogl[all]
```

6.1.3 Installation Methods

From PyPI (Recommended) *Coming Soon!

```
pip install picogl
```

From Source

1. **Clone the repository**:

```
git clone https://github.com/your-org/picogl.git
cd picogl
```

2. **Install in development mode**:

```
pip install -e .
```

3. **Install with all dependencies**:

```
pip install -e .[all]
```

From Conda *Coming Soon!

```
conda install -c conda-forge picogl
```

6.1.4 Dependencies

Core Dependencies

- **PyOpenGL**: Python bindings for OpenGL
- **PyOpenGL-accelerate**: Accelerated OpenGL operations
- **NumPy**: Numerical computing library
- **PyGLM**: OpenGL Mathematics library

Optional Dependencies

- **Pillow**: Image processing for texture loading
- **PyQt5/PyQt6**: Qt-based UI backends
- **GLUT**: GLUT-based examples

Development Dependencies

- **pytest**: Testing framework
- **pytest-cov**: Coverage testing
- **black**: Code formatting
- **flake8**: Linting
- **mypy**: Type checking

6.1.5 Installation Verification

After installation, verify that PicoGL is working correctly:

```python
import picogl
print(f"PicoGL version: {picogl.__version__}")

# Test basic functionality
from picogl.renderer import MeshData
import numpy as np

# Create simple mesh data
vertices = np.array([[0, 0, 0], [1, 0, 0], [0, 1, 0]], dtype=np.float32)
colors = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]], dtype=np.float32)

data = MeshData.from_raw(vertices=vertices, colors=colors)
print(" PicoGL installation successful!")
```

6.1.6 Troubleshooting

Common Issues

ImportError: No module named 'OpenGL'
Install PyOpenGL: `pip install PyOpenGL PyOpenGL-accelerate`

ImportError: No module named 'numpy'
Install NumPy: `pip install numpy`

ImportError: No module named 'pyglm'
Install PyGLM: `pip install pyglm`

OpenGL context creation failed
Update graphics drivers and ensure OpenGL is supported

Segmentation fault on macOS
Run from Terminal.app or iTerm2, not from IDE

Black screen or no rendering
Check OpenGL support and try software rendering

macOS Specific Issues

"No OpenGL context" error

- Run from Terminal.app or iTerm2
- Check if XQuartz is installed and running
- Try software rendering: `export MESA_GL_VERSION_OVERRIDE=3.3`

"Shader compilation failed" error

- Use legacy examples instead
- Check OpenGL version support

"Segmentation fault" error

- Run from Terminal.app
- Check OpenGL drivers
- Try different OpenGL settings

Windows Specific Issues

"Microsoft Visual C++ 14.0 is required"

- Install Visual Studio Build Tools
- Or install pre-compiled wheels

"OpenGL not supported" error

- Update graphics drivers
- Check if OpenGL is enabled in graphics settings

"DLL load failed" error

- Install Visual C++ Redistributable
- Check Python architecture (32-bit vs 64-bit)

Linux Specific Issues

"No display available" error

- Ensure X11 or Wayland is running
- Check DISPLAY environment variable

"OpenGL not supported" error

- Install mesa libraries
- Check graphics drivers

"Permission denied" error

- Run with appropriate permissions
- Check file permissions

6.1.7 Getting Help

If you encounter issues not covered here:

1. **Check the troubleshooting guide** in the documentation
2. **Search existing issues** on GitHub
3. **Create a new issue** with detailed information about your system
4. **Join the community** discussions

6.1.8 System Information

When reporting issues, please include:

- Operating system and version
- Python version
- Graphics card and drivers
- OpenGL version
- Complete error message
- Steps to reproduce the issue

```
import sys
import platform
import OpenGL.GL as GL

print(f"Python: {sys.version}")
print(f"Platform: {platform.platform()}")
print(f"OpenGL: {GL.glGetString(GL.GL_VERSION)}")
print(f"Vendor: {GL.glGetString(GL.GL_VENDOR)}")
print(f"Renderer: {GL.glGetString(GL.GL_RENDERER)}")
```

6.2 Quick Start Guide

This guide will get you up and running with PicoGL in just a few minutes. We'll create a simple colored cube that you can rotate and interact with.

6.2.1 Prerequisites

Before starting, make sure you have:

- Python 3.7 or higher
- PicoGL installed (see *Installation*)
- A system with OpenGL support

6.2.2 Your First PicoGL Program

Let's create a simple program that displays a colored cube:

```
from picogl.renderer import MeshData
from picogl.ui.backend.glut.window.object import RenderWindow
import numpy as np

def main():
    # Define cube vertices (8 corners of a cube)
    vertices = np.array([
        # Front face
        [-1, -1,  1], [ 1, -1,  1], [ 1,  1,  1], [-1,  1,  1],
        # Back face
        [-1, -1, -1], [-1,  1, -1], [ 1,  1, -1], [ 1, -1, -1],
        # Top face
        [-1,  1, -1], [-1,  1,  1], [ 1,  1,  1], [ 1,  1, -1],
        # Bottom face
        [-1, -1, -1], [ 1, -1, -1], [ 1, -1,  1], [-1, -1,  1],
        # Right face
        [ 1, -1, -1], [ 1,  1, -1], [ 1,  1,  1], [ 1, -1,  1],
        # Left face
        [-1, -1, -1], [-1, -1,  1], [-1,  1,  1], [-1,  1, -1]
    ], dtype=np.float32)

    # Define colors for each vertex
    colors = np.array([
        # Front face (red)
        [1, 0, 0], [1, 0, 0], [1, 0, 0], [1, 0, 0],
        # Back face (green)
        [0, 1, 0], [0, 1, 0], [0, 1, 0], [0, 1, 0],
        # Top face (blue)
        [0, 0, 1], [0, 0, 1], [0, 0, 1], [0, 0, 1],
        # Bottom face (yellow)
        [1, 1, 0], [1, 1, 0], [1, 1, 0], [1, 1, 0],
        # Right face (magenta)
        [1, 0, 1], [1, 0, 1], [1, 0, 1], [1, 0, 1],
        # Left face (cyan)
        [0, 1, 1], [0, 1, 1], [0, 1, 1], [0, 1, 1]
    ], dtype=np.float32)

    # Create mesh data
    data = MeshData.from_raw(vertices=vertices, colors=colors)

    # Create render window
    window = RenderWindow(
        width=800,
        height=600,
        title="My First PicoGL Cube",
        data=data
```

(continues on next page)

(continued from previous page)

```
    )

    # Initialize and run
    window.initialize()
    window.run()

if __name__ == "__main__":
    main()
```

Save this code as `my_first_cube.py` and run it:

```
python my_first_cube.py
```

You should see a window with a colorful cube that you can rotate with your mouse!

6.2.3 Understanding the Code

Let's break down what this code does:

1. **Import PicoGL modules**: - `MeshData`: Container for vertex and color data - `RenderWindow`: Window for displaying 3D content
2. **Define vertices**: 24 vertices defining 6 faces of a cube
3. **Define colors**: One color per vertex
4. **Create mesh data**: Combine vertices and colors into a mesh
5. **Create window**: Set up the display window
6. **Run**: Start the interactive display

6.2.4 Key Concepts

MeshData

`MeshData` is the core data structure in PicoGL. It holds all the information needed to render a 3D object:

```
data = MeshData.from_raw(
    vertices=vertices,    # 3D positions
    colors=colors,        # RGB colors
    normals=normals,      # Surface normals (optional)
    uvs=uvs              # Texture coordinates (optional)
)
```

RenderWindow

`RenderWindow` provides the display interface. It handles:

- OpenGL context creation
- Window management
- User input (mouse, keyboard)
- Rendering loop

```
window = RenderWindow(
    width=800,           # Window width
    height=600,          # Window height
    title="My App",      # Window title
    data=mesh_data       # Mesh to display
)
```

6.2.5 Interactive Controls

The default controls are:

- **Mouse drag**: Rotate the view
- **Mouse wheel**: Zoom in/out
- **R key**: Reset rotation
- **W key**: Toggle wireframe mode
- **ESC key**: Exit

6.2.6 Adding More Features

Textures

To add textures to your cube:

```
# Load texture data
from picogl.utils.loader.texture import TextureLoader

texture_loader = TextureLoader("path/to/texture.png")
texture = texture_loader.load()

# Create mesh with UV coordinates
data = MeshData.from_raw(
    vertices=vertices,
    colors=colors,
    uvs=uv_coordinates  # Texture coordinates
)

# Create textured render window
window = TextureWindow(
    width=800,
    height=600,
    title="Textured Cube",
    data=data,
    use_texture=True,
    texture_file="texture.png"
)
```

Lighting

To add lighting effects:

```
# Create mesh with normals
data = MeshData.from_raw(
    vertices=vertices,
    colors=colors,
    normals=normals  # Surface normals for lighting
)
```

Multiple Objects

To render multiple objects:

```
# Create multiple mesh data objects
cube_data = MeshData.from_raw(vertices=cube_vertices, colors=cube_colors)
sphere_data = MeshData.from_raw(vertices=sphere_vertices, colors=sphere_colors)
```

(continues on next page)

(continued from previous page)

```
# Create separate renderers for each object
cube_renderer = ObjectRenderer(context=context, data=cube_data)
sphere_renderer = ObjectRenderer(context=context, data=sphere_data)
```

6.2.7 Legacy Compatibility

If you're having issues with modern OpenGL, try the legacy examples:

```
# Use legacy cube renderer
from examples.legacy_cube_minimal import MinimalCubeRenderer

renderer = MinimalCubeRenderer(width=800, height=600)
renderer.run()
```

6.2.8 Common Issues

Black screen or no rendering

- Check if OpenGL is supported on your system
- Try the legacy examples for better compatibility

Import errors

- Make sure PicoGL is installed: `pip install picogl`
- Check that all dependencies are installed

Window doesn't appear

- Run from Terminal.app (macOS) or command prompt (Windows)
- Check display environment variables

Performance issues

- Reduce the number of vertices
- Use simpler shaders
- Try the legacy rendering mode

6.2.9 Next Steps

Now that you have a basic understanding of PicoGL:

1. **Explore the examples** in the `examples/` directory
2. **Read the API documentation** to learn about all available classes
3. **Try the legacy examples** if you have compatibility issues
4. **Check out the molecular visualization examples** for more complex use cases

6.2.10 Examples to Try

- `examples/cube.py` - Basic colored cube
- `examples/teapot.py` - 3D teapot model
- `examples/legacy_cube_minimal.py` - Legacy-compatible cube
- `examples/molecular_viewer.py` - Molecular visualization

For more advanced examples, see the *Examples* section.

6.2.11 Troubleshooting

If you run into problems:

1. **Check the installation guide** for platform-specific issues
2. **Try the legacy examples** for better compatibility
3. **Check the troubleshooting guide** for common solutions
4. **Report issues** on GitHub with system information

Happy coding with PicoGL!

6.3 Examples

PicoGL comes with a comprehensive set of examples demonstrating various features and use cases. This section provides an overview of all available examples and how to run them.

6.3.1 Basic Examples

Cube Example

File: `examples/cube.py`

A simple colored cube demonstrating basic mesh rendering.

```
from picogl.renderer import MeshData
from picogl.ui.backend.glut.window.object import RenderWindow
from examples.data.cube_data import g_vertex_buffer_data, g_color_buffer_data

data = MeshData.from_raw(vertices=g_vertex_buffer_data, colors=g_color_buffer_data)
window = RenderWindow(width=800, height=600, title="Cube", data=data)
window.initialize()
window.run()
```

Features: * 36 vertices forming 12 triangles * Per-vertex coloring * Interactive rotation and zoom * Modern OpenGL 3.3+ shaders

Run: .. code-block:: bash

python examples/cube.py

Teapot Example

File: `examples/teapot.py`

A 3D teapot model with lighting and shading.

```
from picogl.renderer import MeshData
from picogl.ui.backend.glut.window.object import RenderWindow
from picogl.utils.loader.object import ObjectLoader

# Load teapot data
obj_loader = ObjectLoader("data/teapot.obj")
teapot_data = obj_loader.to_array_style()

data = MeshData.from_raw(
    vertices=teapot_data.vertices,
    normals=teapot_data.normals,
    colors=[[1.0, 0.0, 0.0]] * (len(teapot_data.vertices) // 3)
)
```

(continues on next page)

(continued from previous page)

```
window = RenderWindow(width=800, height=600, title="Teapot", data=data)
window.initialize()
window.run()
```

Features: * Loaded from OBJ file * Surface normals for lighting * Phong shading * Interactive controls

Run: .. code-block:: bash

python examples/teapot.py

6.3.2 Legacy Examples

For systems with limited OpenGL support (older macOS, etc.), use the legacy examples:

Legacy Cube (Minimal)

File: `examples/legacy_cube_minimal.py`

Maximum compatibility cube renderer using only PyOpenGL.

Features: * OpenGL 1.x immediate mode * No external dependencies beyond PyOpenGL * Works on any system with basic OpenGL support * Same visual appearance as modern version

Run: .. code-block:: bash

python examples/legacy_cube_minimal.py

Legacy Cube (Fixed)

File: `examples/legacy_cube_fixed.py`

PicoGL-integrated legacy cube using LegacyGLMesh.

Features: * OpenGL 1.x/2.x compatibility * Uses PicoGL LegacyGLMesh * Falls back to wireframe if mesh loading fails * Better integration with PicoGL ecosystem

Run: .. code-block:: bash

python examples/legacy_cube_fixed.py

Legacy Teapot (Minimal)

File: `examples/legacy_teapot_minimal.py`

Maximum compatibility teapot using built-in OpenGL primitives.

Features: * Uses `glutSolidTeapot()` built-in primitive * No external files required * Maximum compatibility * Interactive controls

Run: .. code-block:: bash

python examples/legacy_teapot_minimal.py

Legacy Teapot (Fixed)

File: `examples/legacy_teapot_fixed.py`

PicoGL-integrated legacy teapot with OBJ file support.

Features: * Loads teapot from OBJ file * Uses LegacyGLMesh for rendering * Falls back to wireframe teapot if OBJ loading fails * Full PicoGL integration

Run: .. code-block:: bash

python examples/legacy_teapot_fixed.py

6.3.3 Advanced Examples

Molecular Visualization

File: `examples/molecular_viewer.py`

Complex molecular visualization with atom and bond rendering.

Features: * PDB file loading * Atom and bond rendering * Multiple rendering modes * Interactive controls * Scientific visualization

Run: .. code-block:: bash

python examples/molecular_viewer.py

Texture Examples

File: `examples/texture.py`

Demonstrates texture mapping and UV coordinates.

Features: * Texture loading from image files * UV coordinate mapping * Multiple texture formats * Texture filtering and wrapping

Run: .. code-block:: bash

python examples/texture.py

Shader Examples

Directory: `examples/glsl/`

Various shader examples demonstrating different rendering techniques:

- `tu01/` - Basic color cube
- `tu02/` - Texture mapping
- `tu04/` - Lighting and shading
- `tu07/` - Standard shading
- `tu08/` - Transparent shading
- `tu09/` - Text rendering
- `tu10/` - Normal mapping

Run: .. code-block:: bash

Run specific shader example python examples/tu_01_color_cube.py python examples/tu_02_texture_without_normal.py python examples/tu_04_lighting.py

6.3.4 Utility Examples

Mesh Viewer

File: `examples/utils/meshViewer.py`

Generic mesh viewer for OBJ files.

Features: * Load any OBJ file * Interactive viewing * Multiple rendering modes * Export capabilities

Run: .. code-block:: bash

python examples/utils/meshViewer.py path/to/model.obj

Test Window

File: `examples/utils/test_window.py`

Simple test window for debugging.

Features: * Basic OpenGL context testing * Simple rendering * Debug information * Error reporting

Run: .. code-block:: bash

python examples/utils/test_window.py

6.3.5 Running Examples

Prerequisites

Before running examples, ensure you have:

1. **PicoGL installed** (see *Installation*)
2. **Required dependencies**: - PyOpenGL - NumPy - PyGLM - Pillow (for texture examples)
3. **OpenGL support** on your system

Basic Usage

Most examples can be run directly:

```
# Navigate to the examples directory
cd examples

# Run a basic example
python cube.py

# Run a legacy example
python legacy_cube_minimal.py
```

Interactive Controls

Most examples support these controls:

- **Mouse drag**: Rotate the view
- **Mouse wheel**: Zoom in/out
- **R key**: Reset rotation
- **W key**: Toggle wireframe mode
- **F key**: Fill mode
- **ESC key**: Exit

Some examples have additional controls:

- **N key**: Toggle normals display
- **Space**: Toggle auto-rotation
- **+/-**: Zoom in/out

6.3.6 Troubleshooting Examples

Common Issues

"No OpenGL context" error

- Try legacy examples instead

- Run from Terminal.app (macOS) or command prompt (Windows)
- Check display environment

"Shader compilation failed" error

- Use legacy examples (no shaders required)
- Check OpenGL version support
- Update graphics drivers

"Import error" for PicoGL modules

- Install PicoGL: `pip install picogl`
- Check Python path and virtual environment

"File not found" error for data files

- Ensure you're running from the examples directory
- Check that data files exist in the correct location

Black screen or no rendering

- Try legacy examples for better compatibility
- Check OpenGL support
- Update graphics drivers

Platform-Specific Issues

macOS

Segmentation fault

- Run from Terminal.app or iTerm2
- Check OpenGL drivers
- Try software rendering

"No display available" error

- Install XQuartz
- Check DISPLAY environment variable

Windows

"DLL load failed" error

- Install Visual C++ Redistributable
- Check Python architecture (32-bit vs 64-bit)

"OpenGL not supported" error

- Update graphics drivers
- Check OpenGL support in graphics settings

Linux

"No display available" error

- Ensure X11 or Wayland is running
- Check DISPLAY environment variable

"Permission denied" error

- Run with appropriate permissions
- Check file permissions

6.3.7 Example Selection Guide

Choose the right example for your needs:

Learning OpenGL basics

- Start with `cube.py` or `legacy_cube_minimal.py`
- Try `teapot.py` for more complex geometry

Compatibility issues

- Use `legacy_*_minimal.py` examples
- These work on any system with basic OpenGL support

Advanced features

- Try `molecular_viewer.py` for complex rendering
- Explore shader examples in `glsl/` directory

Texture mapping

- Use `texture.py` or `tu02/` shader example
- Check `legacy_teapot_fixed.py` for legacy texture support

Scientific visualization

- Use `molecular_viewer.py`
- Check `examples/README_MOLECULAR.md` for detailed guide

6.3.8 Customizing Examples

You can modify examples to suit your needs:

1. **Change colors**: Modify the color arrays
2. **Add more objects**: Create additional MeshData objects
3. **Change lighting**: Modify shader parameters
4. **Add textures**: Load and apply texture images
5. **Modify controls**: Change keyboard/mouse bindings

For more information, see the *Renderer API* documentation.

6.3.9 Contributing Examples

We welcome contributions of new examples! When contributing:

1. **Follow the existing style** and structure
2. **Include comprehensive comments** explaining the code
3. **Test on multiple platforms** if possible
4. **Provide both modern and legacy versions** when appropriate
5. **Update this documentation** to include your example

For more information, see the *Contributing* guide.

6.4 Legacy Examples

PicoGL includes comprehensive legacy examples designed to work on systems with limited OpenGL support, including older macOS systems and systems without modern shader support.

6.4.1 Overview

The legacy examples provide multiple fallback renderers that work on systems with limited OpenGL support:

- **Minimal versions**: Use only PyOpenGL, maximum compatibility
- **Fixed versions**: Use PicoGL with legacy rendering, medium compatibility
- **Original versions**: Use modern OpenGL 3.3+, low compatibility

6.4.2 Why Legacy Examples?

Modern OpenGL (3.3+) requires: * Shader compilation support * Vertex Array Objects (VAOs) * Modern graphics drivers * Specific OpenGL context requirements

Legacy examples work on: * Older macOS systems * Systems without shader support * Systems with limited OpenGL drivers * Headless environments (with proper setup) * Educational environments with restricted OpenGL

6.4.3 Legacy Teapot Examples

Legacy Teapot (Minimal)

File: `examples/legacy_teapot_minimal.py`

The most compatible teapot renderer, using only built-in OpenGL primitives.

Features: * Uses `glutSolidTeapot()` built-in primitive * No external dependencies beyond PyOpenGL * OpenGL 1.x immediate mode rendering * Interactive rotation and zoom * Multiple rendering modes

Controls: * Mouse drag: Rotate view * Mouse wheel: Zoom in/out * R: Reset rotation * W: Toggle wireframe mode * F: Fill mode * +/-: Zoom in/out * Space: Toggle auto-rotation * ESC: Exit

Run: .. code-block:: bash

> python examples/legacy_teapot_minimal.py

Compatibility: Maximum (works on any OpenGL system)

Legacy Teapot (Fixed)

File: `examples/legacy_teapot_fixed.py`

PicoGL-integrated legacy teapot with OBJ file support.

Features: * Uses `LegacyGLMesh` for OpenGL 1.x/2.x compatibility * Loads teapot data from OBJ file * Falls back to wireframe teapot if OBJ loading fails * Full PicoGL integration * Interactive controls

Controls: * Mouse drag: Rotate view * Mouse wheel: Zoom in/out * R: Reset rotation * W: Toggle wireframe mode * F: Fill mode * +/-: Zoom in/out * Space: Toggle auto-rotation * ESC: Exit

Run: .. code-block:: bash

> python examples/legacy_teapot_fixed.py

Compatibility: Medium (requires PicoGL library)

6.4.4 Legacy Cube Examples

Legacy Cube (Minimal)

File: `examples/legacy_cube_minimal.py`

The most compatible cube renderer, using immediate mode OpenGL.

Features: * Uses immediate mode OpenGL (`glBegin/glEnd`) * No external dependencies beyond PyOpenGL * Same vertex and color data as original cube * Interactive rotation and zoom * Multiple rendering modes * Normal vector display

Controls: * Mouse drag: Rotate view * Mouse wheel: Zoom in/out * R: Reset rotation * W: Toggle wireframe mode * F: Fill mode * N: Toggle normals display * +/-: Zoom in/out * Space: Toggle auto-rotation * ESC: Exit

Run: .. code-block:: bash

python examples/legacy_cube_minimal.py

Compatibility: Maximum (works on any OpenGL system)

Legacy Cube (Fixed)

File: `examples/legacy_cube_fixed.py`

PicoGL-integrated legacy cube using LegacyGLMesh.

Features: * Uses `LegacyGLMesh` for OpenGL 1.x/2.x compatibility * Loads cube data using PicoGL MeshData * Falls back to wireframe cube if mesh loading fails * Full PicoGL integration * Interactive controls

Controls: * Mouse drag: Rotate view * Mouse wheel: Zoom in/out * R: Reset rotation * W: Toggle wireframe mode * F: Fill mode * +/-: Zoom in/out * Space: Toggle auto-rotation * ESC: Exit

Run: .. code-block:: bash

python examples/legacy_cube_fixed.py

Compatibility: Medium (requires PicoGL library)

6.4.5 Diagnostic Tools

OpenGL Setup Test

File: `examples/test_opengl_setup.py`

Comprehensive diagnostic tool for OpenGL setup issues.

Features: * Tests OpenGL imports * Tests display environment * Tests OpenGL context creation * Tests PicoGL imports * Provides troubleshooting guidance

Run: .. code-block:: bash

python examples/test_opengl_setup.py

Output: Detailed report of OpenGL capabilities and issues

Cube Data Test

File: `examples/test_cube_data.py`

Tests cube data structure and PicoGL imports without requiring a display.

Features: * Validates cube vertex and color data * Tests triangle structure * Tests PicoGL imports * Tests mesh creation * Provides troubleshooting guidance

Run: .. code-block:: bash

python examples/test_cube_data.py

Output: Validation report of cube data and imports

6.4.6 Installation and Setup

Prerequisites

Minimal Examples: * Python 3.7+ * PyOpenGL * GLUT (usually included with PyOpenGL)

Fixed Examples: * Python 3.7+ * PyOpenGL * GLUT * NumPy * PicoGL library

Installation

```
# Install minimal requirements
pip install PyOpenGL PyOpenGL_accelerate numpy

# For fixed examples (PicoGL integration)
pip install picogl
```

Platform-Specific Setup

macOS

1. **Install XQuartz** (for X11 support): .. code-block:: bash

   brew install –cask xquartz

2. **Run from Terminal.app or iTerm2** (not from IDE)
3. **Check OpenGL support**: .. code-block:: python

   import OpenGL.GL as GL print(GL.glGetString(GL.GL_VERSION))

Windows

1. **Update graphics drivers** to latest version
2. **Install Visual C++ Redistributable** if needed
3. **Run from command prompt** (not from IDE)

Linux

1. **Install OpenGL libraries**: .. code-block:: bash

   # Ubuntu/Debian sudo apt install libgl1-mesa-dev libglu1-mesa

   # CentOS/RHEL sudo yum install mesa-libGL mesa-libGLU

2. **Ensure X11 or Wayland is running**
3. **Check display environment**: .. code-block:: bash

   echo $DISPLAY

6.4.7 Troubleshooting

Common Issues

"No OpenGL context" error

- Use minimal versions instead
- Run from Terminal.app (macOS) or command prompt (Windows)
- Check display environment

"Shader compilation failed" error

- Use minimal versions (no shaders required)
- Check OpenGL version support
- Update graphics drivers

"PicoGL import failed" error

- Use minimal versions (no PicoGL required)
- Or install PicoGL: `pip install picogl`

“Segmentation fault” error

- Use minimal versions
- Check OpenGL drivers
- Try software rendering

“Black screen” error

- Check OpenGL support
- Use minimal versions
- Try different OpenGL settings

“No display available” error

- Check display environment
- Install XQuartz (macOS)
- Ensure X11/Wayland is running (Linux)

macOS Specific Issues

OpenGL context creation fails

- Run from Terminal.app or iTerm2
- Check if XQuartz is installed and running
- Try software rendering: `export MESA_GL_VERSION_OVERRIDE=3.3`

Segmentation fault

- Run from Terminal.app
- Check OpenGL drivers
- Try different OpenGL settings

“No display available” error

- Install XQuartz: `brew install --cask xquartz`
- Check DISPLAY environment variable
- Restart XQuartz

Windows Specific Issues

“DLL load failed” error

- Install Visual C++ Redistributable
- Check Python architecture (32-bit vs 64-bit)

“OpenGL not supported” error

- Update graphics drivers
- Check OpenGL support in graphics settings

“Microsoft Visual C++ 14.0 is required”

- Install Visual Studio Build Tools
- Or install pre-compiled wheels

Linux Specific Issues

"No display available" error

- Ensure X11 or Wayland is running
- Check DISPLAY environment variable

"OpenGL not supported" error

- Install mesa libraries
- Check graphics drivers

"Permission denied" error

- Run with appropriate permissions
- Check file permissions

6.4.8 Performance Notes

Minimal Versions: * Fastest performance * Uses hardware-optimized primitives * Immediate mode rendering * No shader compilation overhead

Fixed Versions: * Medium performance * Uses legacy VBOs * Some shader compilation * Better integration with PicoGL

Original Versions: * Slowest performance * Full shader pipeline * Modern VAO/VBO usage * Maximum feature set

6.4.9 Choosing the Right Version

Use Minimal Versions When: * Maximum compatibility is required * OpenGL support is limited * Performance is critical * No PicoGL integration needed

Use Fixed Versions When: * PicoGL integration is desired * Some OpenGL support is available * Balance between compatibility and features * OBJ file loading is needed

Use Original Versions When: * Full OpenGL 3.3+ support is available * Maximum features are needed * Modern shader pipeline is required * Best visual quality is desired

6.4.10 Development Notes

Adding New Legacy Examples:

1. **Follow the naming convention**: `legacy_<name>_<type>.py`
2. **Provide both minimal and fixed versions** when possible
3. **Include comprehensive error handling**
4. **Test on multiple platforms**
5. **Document compatibility requirements**

Example Structure: .. code-block:: python

class LegacyExampleRenderer:

def __init__(self, width=800, height=600, title="Example"):
Initialize renderer

def init_glut(self):
Initialize GLUT window

def init_gl(self):
Initialize OpenGL state

def display(self):
Render the scene

def run(self):
Main loop

Error Handling: .. code-block:: python

try:
OpenGL operations

except Exception as e:
print(f'Warning: OpenGL issue: {e}") # Fallback behavior

For more information, see the *Contributing* guide.

6.4.11 License

The legacy examples are part of the PicoGL project and follow the same license terms.

6.5 Troubleshooting

This guide helps you diagnose and fix common issues with PicoGL.

6.5.1 Common Issues

Installation Problems

"No module named 'picogl'" error

- Ensure PicoGL is installed: `pip install picogl`
- Check Python path and virtual environment
- Verify installation: `python -c "import picogl"`

"No module named 'OpenGL'" error

- Install PyOpenGL: `pip install PyOpenGL PyOpenGL-accelerate`
- Check OpenGL installation on your system

"No module named 'numpy'" error

- Install NumPy: `pip install numpy`
- Check NumPy version compatibility

"No module named 'pyglm'" error

- Install PyGLM: `pip install pyglm`
- Check PyGLM version compatibility

"No module named 'PIL'" error

- Install Pillow: `pip install Pillow`
- Check Pillow version compatibility

OpenGL Context Issues

"No OpenGL context" error

- Run from Terminal.app (macOS) or command prompt (Windows)
- Check display environment variables
- Verify OpenGL drivers are installed

"OpenGL context creation failed" error

- Update graphics drivers
- Check OpenGL support on your system
- Try software rendering

"Segmentation fault" error

- Run from Terminal.app (macOS)
- Check OpenGL drivers
- Try different OpenGL settings

"Black screen" or no rendering

- Check OpenGL support
- Try legacy examples for better compatibility
- Update graphics drivers

Shader Compilation Issues

"Shader compilation failed" error

- Use legacy examples (no shaders required)
- Check OpenGL version support
- Update graphics drivers

"Shader link failed" error

- Check shader compatibility
- Verify shader source code
- Check OpenGL version

"Uniform not found" error

- Check uniform names in shaders
- Verify uniform types
- Check shader program binding

"Texture loading failed" error

- Check texture file format
- Verify texture file path
- Check OpenGL texture support

6.5.2 Platform-Specific Issues

macOS Issues

"No display available" error

- Install XQuartz: `brew install --cask xquartz`
- Check DISPLAY environment variable
- Restart XQuartz

"OpenGL not supported" error

- Check OpenGL version: `python -c "import OpenGL.GL as GL; print(GL.glGetString(GL.GL_VERSION))"`

- Update graphics drivers
- Try software rendering

"Segmentation fault" error

- Run from Terminal.app or iTerm2
- Check OpenGL drivers
- Try different OpenGL settings

"Shader compilation failed" error

- Use legacy examples instead
- Check OpenGL version support
- Try software rendering

Troubleshooting Steps: 1. Run from Terminal.app or iTerm2 2. Check OpenGL version and support 3. Install XQuartz if needed 4. Try legacy examples for compatibility 5. Update graphics drivers

Windows Issues

"DLL load failed" error

- Install Visual C++ Redistributable
- Check Python architecture (32-bit vs 64-bit)
- Reinstall PyOpenGL

"OpenGL not supported" error

- Update graphics drivers
- Check OpenGL support in graphics settings
- Try software rendering

"Microsoft Visual C++ 14.0 is required" error

- Install Visual Studio Build Tools
- Or install pre-compiled wheels

"Permission denied" error

- Run as administrator
- Check file permissions
- Check antivirus software

Troubleshooting Steps: 1. Update graphics drivers 2. Install Visual C++ Redistributable 3. Check Python architecture 4. Try running as administrator 5. Check antivirus software

Linux Issues

"No display available" error

- Ensure X11 or Wayland is running
- Check DISPLAY environment variable
- Install display server

"OpenGL not supported" error

- Install mesa libraries: `sudo apt install libgl1-mesa-dev libglu1-mesa`
- Check graphics drivers
- Try software rendering

"Permission denied" error

- Run with appropriate permissions
- Check file permissions
- Check SELinux/AppArmor

"Library not found" error

- Install required libraries
- Check library paths
- Update package manager

Troubleshooting Steps: 1. Install mesa libraries 2. Check display server 3. Update graphics drivers 4. Check library paths 5. Try software rendering

6.5.3 Performance Issues

Slow Rendering

Causes: * Too many draw calls * Inefficient shaders * Large textures * Poor buffer management

Solutions: * Use instanced rendering * Optimize shaders * Use appropriate texture sizes * Batch similar operations

Example: .. code-block:: python

> # Inefficient rendering for obj in objects:
>
> > obj.render() # Many draw calls
>
> # Efficient rendering batch_objects(objects) render_batch() # Single draw call

Memory Issues

Causes: * Large textures * Too many buffers * Memory leaks * Inefficient data structures

Solutions: * Use appropriate texture sizes * Clean up unused buffers * Monitor memory usage * Use efficient data types

Example: .. code-block:: python

> # Memory leak def create_mesh():
>
> > vao = VertexArrayObject() # … use vao … # Forgot to delete vao
>
> # Fixed def create_mesh():
>
> > vao = VertexArrayObject() try:
> >
> > > # … use vao …
> >
> > **finally:**
> > vao.delete()

Frame Rate Issues

Causes: * VSync enabled * Too many operations per frame * Inefficient rendering * Poor OpenGL state management

Solutions: * Disable VSync if needed * Optimize rendering loop * Use efficient rendering techniques * Minimize state changes

Example: .. code-block:: python

> # Inefficient rendering def render():
>
> > glEnable(GL_DEPTH_TEST) draw_object1() glDisable(GL_DEPTH_TEST) glEnable(GL_DEPTH_TEST) draw_object2()

Efficient rendering def render():

glEnable(GL_DEPTH_TEST) draw_object1() draw_object2() glDisable(GL_DEPTH_TEST)

6.5.4 Debugging Techniques

Enable Debug Output

OpenGL Error Checking: .. code-block:: python

def check_gl_error():
error = glGetError() if error != GL_NO_ERROR:

print(f"OpenGL error: {error}") return False

return True

Use after OpenGL calls glDrawElements(GL_TRIANGLES, count, GL_UNSIGNED_INT, None) check_gl_error()

Debug Information: .. code-block:: python

def print_gl_info():
print(f"OpenGL version: {glGetString(GL_VERSION)}") print(f"Vendor: {glGetString(GL_VENDOR)}") print(f"Renderer: {glGetString(GL_RENDERER)}") print(f"Extensions: {glGetString(GL_EXTENSIONS)}")

Performance Profiling: .. code-block:: python

import time

def profile_render():
start_time = time.time() self.render_scene() end_time = time.time() frame_time = end_time - start_time fps = 1.0 / frame_time print(f"Frame time: {frame_time:.3f}s, FPS: {fps:.1f}")

Use Diagnostic Tools

OpenGL Setup Test: .. code-block:: bash

python examples/test_opengl_setup.py

Cube Data Test: .. code-block:: bash

python examples/test_cube_data.py

Legacy Examples: .. code-block:: bash

Try minimal examples for compatibility python examples/legacy_cube_minimal.py python examples/legacy_teapot_minimal.py

Check System Information

Python Information: .. code-block:: python

import sys import platform print(f"Python: {sys.version}") print(f"Platform: {platform.platform()}") print(f"Architecture: {platform.architecture()}")

OpenGL Information: .. code-block:: python

import OpenGL.GL as GL print(f"OpenGL version: {GL.glGetString(GL.GL_VERSION)}") print(f"Vendor: {GL.glGetString(GL.GL_VENDOR)}") print(f"Renderer: {GL.glGetString(GL.GL_RENDERER)}")

Package Information: .. code-block:: python

import pkg_resources packages = ['picogl', 'numpy', 'PyOpenGL', 'pyglm', 'Pillow'] for package in packages:

try:
version = pkg_resources.get_distribution(package).version print(f"{package}: {version}")

except pkg_resources.DistributionNotFound:
print(f"{package}: Not installed")

6.5.5 Common Solutions

Try Legacy Examples

If modern examples don't work, try legacy examples:

```
# Minimal examples (maximum compatibility)
python examples/legacy_cube_minimal.py
python examples/legacy_teapot_minimal.py

# Fixed examples (PicoGL integration)
python examples/legacy_cube_fixed.py
python examples/legacy_teapot_fixed.py
```

Update Dependencies

Update all dependencies to latest versions:

```
pip install --upgrade picogl numpy PyOpenGL PyOpenGL-accelerate pyglm Pillow
```

Check OpenGL Support

Verify OpenGL support on your system:

```
import OpenGL.GL as GL

# Check OpenGL version
version = GL.glGetString(GL.GL_VERSION)
print(f"OpenGL version: {version}")

# Check extensions
extensions = GL.glGetString(GL.GL_EXTENSIONS)
print(f"Extensions: {extensions}")
```

Use Software Rendering

If hardware rendering fails, try software rendering:

```
# Set environment variable
export MESA_GL_VERSION_OVERRIDE=3.3
python examples/cube.py
```

Check Display Environment

Verify display environment:

```
# Check display
echo $DISPLAY

# Check X11
xdpyinfo
```

(continues on next page)

(continued from previous page)

```
# Check Wayland
echo $WAYLAND_DISPLAY
```

6.5.6 Report Issues

When reporting issues, include:

1. **System Information**: - Operating system and version - Python version - Graphics card and drivers - OpenGL version
2. **Error Information**: - Complete error message - Steps to reproduce - Expected vs actual behavior
3. **Environment Information**: - Virtual environment details - Package versions - Display environment
4. **Code Information**: - Minimal code to reproduce - Full traceback - Relevant configuration

Example Issue Report: .. code-block:: text

> **System**: macOS 12.0, Python 3.9, Intel HD Graphics 6000 **Error**: "No OpenGL context" when running cube.py **Steps**: 1. Install PicoGL: pip install picogl 2. Run: python examples/cube.py 3. Get error: "No OpenGL context" **Expected**: Cube window should appear **Actual**: Error message and no window **Code**: Standard cube.py example **Environment**: Terminal.app, XQuartz installed

6.5.7 Getting Help

Documentation: * Check this troubleshooting guide * Read the API documentation * Look at example code

Community: * Search existing issues on GitHub * Create a new issue with detailed information * Join community discussions

Professional Support: * Contact maintainers directly * Consider commercial support options * Hire OpenGL experts for complex issues

Resources: * OpenGL documentation * PyOpenGL documentation * Platform-specific OpenGL guides * Graphics driver documentation

Remember: Most issues can be resolved by using the appropriate legacy examples for your system's OpenGL capabilities.

6.6 Renderer API

The renderer module provides the core rendering functionality for PicoGL, including mesh data management, OpenGL context handling, and various renderer implementations.

6.6.1 Core Classes

MeshData

class `picogl.renderer.meshdata.`**`MeshData`**(*vbo: ndarray = None, nbo: ndarray = None, uvs: ndarray = None, cbo: ndarray = None, ebo: ndarray = None*)

Bases: `object`

Holds OpenGL-related state objects for rendering.

`__init__`(*vbo: ndarray = None, nbo: ndarray = None, uvs: ndarray = None, cbo: ndarray = None, ebo: ndarray = None*)

set up the OpenGL context

`as_ribbon_args`() → dict

Convert into arguments for setup_ribbon_buffers.

`bind`()

`unbind()`

`classmethod from_raw`(*vertices: ndarray*, *normals: ndarray = None*, *uvs: ndarray = None*, *colors: ndarray = None*, *indices: ndarray = None*, *color_per_vertex: ndarray = None*)

Build a MeshData from raw/python inputs.

Parameters

- **vertices** – np.ndarray required, list/array of x,y,z triplets
- **normals** – np.ndarray optional, list/array of x,y,z triplets
- **uvs** – np.ndarray optional, list/array of u,v pairs
- **indices** – np.ndarray optional int indices
- **colors** – np.ndarray optional per-vertex colors (flat float32 array)
- **color_per_vertex** – np.ndarray if provided and colors is None, generate per-vertex colors

`draw`(*color: tuple = None*, *line_width: float = 1.0*, *mode: int = OpenGL.GL.GL_TRIANGLES*, *fill: bool = False*, *alpha: float = 1.0*)

Draw the mesh with optional color override and transparency.

Parameters

- **color** – Optional color override. If None and vertex colors exist, uses vertex colors.
- **line_width** – Line width for wireframe mode
- **mode** – OpenGL drawing mode
- **fill** – Whether to fill or use wireframe
- **alpha** – Transparency value from 0.0 (opaque) to 1.0 (fully transparent)

The `MeshData` class is the central data structure for storing 3D mesh information including vertices, colors, normals, and texture coordinates.

Example: .. code-block:: python

from picogl.renderer import MeshData import numpy as np

Create mesh data vertices = np.array([[0, 0, 0], [1, 0, 0], [0, 1, 0]], dtype=np.float32) colors = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]], dtype=np.float32)

data = MeshData.from_raw(vertices=vertices, colors=colors)

GLContext

`class picogl.renderer.glcontext.GLContext`(*vaos: dict[str, ~picogl.backend.modern.core.vertex.array.object.VertexArrayObject] = <factory>*, *vertex_array: ~picogl.backend.modern.core.vertex.array.object.VertexArrayObject | None = None*, *shader: ~picogl.backend.modern.core.shader.program.ShaderProgram | None = None*, *texture_id: int | None = None*, *mvp_matrix: ~numpy.ndarray = <factory>*, *model_matrix: ~numpy.ndarray = <factory>*, *view: ~numpy.ndarray = <factory>*, *eye_np: ~numpy.ndarray = <factory>*)

Bases: `object`

Stores dynamic OpenGL-related state (VAO, shader, texture handles, etc.). Does NOT store raw vertex data.

vaos: dict[str, *VertexArrayObject*]

vertex_array: *VertexArrayObject* | None = None

shader: *ShaderProgram* | None = None

texture_id: int | None = None

mvp_matrix: ndarray

model_matrix: ndarray

view: ndarray

eye_np: ndarray

create_shader_program(*vertex_source_file: str, fragment_source_file: str, glsl_dir: str | Path | None = None*) → None

Parameters

- **vertex_source_file** – str
- **fragment_source_file** – str
- **glsl_dir** – str

Returns
None

__init__(*vaos: dict[str, ~picogl.backend.modern.core.vertex.array.object.VertexArrayObject] = <factory>, vertex_array: ~picogl.backend.modern.core.vertex.array.object.VertexArrayObject | None = None, shader: ~picogl.backend.modern.core.shader.program.ShaderProgram | None = None, texture_id: int | None = None, mvp_matrix: ~numpy.ndarray = <factory>, model_matrix: ~numpy.ndarray = <factory>, view: ~numpy.ndarray = <factory>, eye_np: ~numpy.ndarray = <factory>*) → None

The GLContext class manages OpenGL-related state including VAOs, shaders, textures, and transformation matrices.

Example: .. code-block:: python

from picogl.renderer import GLContext

Create OpenGL context context = GLContext()

Create shader program context.create_shader_program(

vertex_source_file="vertex.glsl", fragment_source_file="fragment.glsl"

)

6.6.2 Renderer Classes

ObjectRenderer

class picogl.renderer.object.**ObjectRenderer**(*context: GLContext, data: MeshData, base_dir: str | Path | None = None, glsl_dir: str | Path | None = None, use_texture: bool = False, texture_file: str | None = None, resource_subdir: str = 'tu02'*)

Bases: *RendererBase*

Unified renderer for textured and untextured objects.

__init__(*context:* GLContext, *data:* MeshData, *base_dir:* *str* | *Path* | *None* = *None*, *glsl_dir:* *str* | *Path* | *None* = *None*, *use_texture:* *bool* = *False*, *texture_file:* *str* | *None* = *None*, *resource_subdir:* *str* = *'tu02'*)

Initialize the renderer.

Parameters
state – Application state object for accessing shared data.

initialize_shaders()

Load and compile shaders.

initialize()

Create VAO and VBOs once.

render() → None

Dispatch render pass.

The `ObjectRenderer` class provides unified rendering for both textured and untextured objects using modern OpenGL.

Example: .. code-block:: python

from picogl.renderer import GLContext, MeshData from picogl.renderer.object import ObjectRenderer

Create renderer context = GLContext() data = MeshData.from_raw(vertices=vertices, colors=colors)

renderer = ObjectRenderer(
context=context, data=data, use_texture=False

)

Initialize and render renderer.initialize_shaders() renderer.initialize() renderer.render()

TextureRenderer

class picogl.renderer.texture.TextureRenderer(*context:* GLContext, *data:* MeshData, *base_dir:* *str* | *Path* = *None*, *glsl_dir:* *str* | *Path* = *None*, *use_texture:* *bool* = *False*, *texture_file:* *str* = *None*, *resource_subdir:* *str* = *None*)

Bases: `ObjectRenderer`

Basic renderer class

__init__(*context:* GLContext, *data:* MeshData, *base_dir:* *str* | *Path* = *None*, *glsl_dir:* *str* | *Path* = *None*, *use_texture:* *bool* = *False*, *texture_file:* *str* = *None*, *resource_subdir:* *str* = *None*)

Initialize the renderer.

Parameters
state – Application state object for accessing shared data.

initialize_textures()

get_texture_filename()

get texture filename

Returns
texture filename: str

set_texture_filename(*file_name:* *str* = *None*)

Parameters
file_name – str = None

Returns
None

set_resource_path(*base_path: str | Path*, *subdir: str*)

Parameters

- **base_path** – str | Path
- **subdir** – str

The `TextureRenderer` class extends `ObjectRenderer` to provide specialized texture rendering capabilities.

Example: .. code-block:: python

from picogl.renderer.texture import TextureRenderer

Create texture renderer renderer = TextureRenderer(

context=context, data=data, base_dir="path/to/resources", use_texture=True, texture_file="texture.png"

)

Initialize textures renderer.initialize_textures()

LegacyGLMesh

The `LegacyGLMesh` class provides OpenGL 1.x/2.x compatible mesh rendering for systems without modern OpenGL support.

Example: .. code-block:: python

from picogl.renderer.legacy_glmesh import LegacyGLMesh

Create legacy mesh mesh = LegacyGLMesh(

vertices=vertices, faces=faces, colors=colors, normals=normals

)

Upload to GPU and draw mesh.upload() mesh.draw()

GLMesh

class picogl.renderer.glmesh.GLMesh(*vertices: ndarray*, *faces: ndarray*, *colors: ndarray | None = None*, *normals: ndarray | None = None*, *uvs: ndarray | None = None*)

Bases: `object`

GPU-resident mesh: owns VAO/VBO/EBO/CBO/NBO for an indexed triangle mesh. It does not know anything about shaders or matrices.

__init__(*vertices: ndarray*, *faces: ndarray*, *colors: ndarray | None = None*, *normals: ndarray | None = None*, *uvs: ndarray | None = None*)

classmethod from_mesh_data(*mesh: MeshData*) → *GLMesh*

Construct a GLMesh from a MeshData container.

Parameters

mesh (`MeshData`) – Must have .vbo (Nx3), .ebo (Mx1), optional .cbo (Nx3), .nbo (Nx3), uvs (Nx2)

Returns

Ready-to-upload mesh (GPU buffers are allocated only when *upload()* is called).

Return type

GLMesh

upload() → None

Allocate & fill GPU buffers.

bind()

unbind()

delete()

Free GPU resources.

draw() → None

Draw the mesh.

The `GLMesh` class provides modern OpenGL mesh rendering with VAO/VBO support.

Example: .. code-block:: python

from picogl.renderer.glmesh import GLMesh

Create modern mesh mesh = GLMesh(

vertices=vertices, faces=faces, colors=colors, normals=normals

)

Upload to GPU and draw mesh.upload() mesh.draw()

6.6.3 Base Classes

RendererBase

`class picogl.renderer.base.`**RendererBase**(*parent: object = None*)

Bases: `AbstractRenderer`

Base Renderer Class

__init__(*parent: object = None*)

Initialize the renderer.

Parameters

state – Application state object for accessing shared data.

property dispatch_list

property initialized: bool

render(*mvp_matrix: ndarray | None = None*) → None

render dispatcher :return: None

initialize() → None

initialize_rendering_buffers

Returns

initialize_rendering_buffers()

For back compatibility

set_visibility(*visible: bool*) → None

Set the visibility of the object.

The `RendererBase` class provides the base functionality for all renderers in PicoGL.

AbstractRenderer

`class picogl.renderer.abstract.`**AbstractRenderer**

Bases: `ABC`

abstractmethod initialize() → None

Subclasses must implement buffer setup.

set_visibility(*visible: bool*) → None

Set the visibility of the object.

abstractmethod render()

The `AbstractRenderer` class defines the interface that all renderers must implement.

6.6.4 Utility Classes

UvRenderer

class picogl.renderer.uvrenderer.**UvRenderer**(*parent: object = None*, *vertex_shader_file: str = 'glsl/utils/uv2d/vertex.glsl'*, *fragment_shader_file: str = 'glsl/utils/uv2d/fragment.glsl'*)

Bases: `RendererBase`

2D UV Renderer that draws a mesh using UV coordinates and indices. Follows the RendererBase interface.

__init__(*parent: object = None*, *vertex_shader_file: str = 'glsl/utils/uv2d/vertex.glsl'*, *fragment_shader_file: str = 'glsl/utils/uv2d/fragment.glsl'*)

Initialize the renderer.

Parameters

state – Application state object for accessing shared data.

initialize(*uv_buffer: int*, *indices_buffer: int*, *index_count: int*) → None

Bind the UV and index buffers to the renderer.

Parameters

- **uv_buffer** – OpenGL buffer ID containing UVs
- **indices_buffer** – OpenGL buffer ID containing indices
- **index_count** – Number of indices to draw

The `UvRenderer` class provides 2D UV coordinate rendering for texture visualization.

Example: .. code-block:: python

from picogl.renderer.uvrenderer import UvRenderer

Create UV renderer renderer = UvRenderer()

Initialize with UV data renderer.initialize(uv_buffer, indices_buffer, index_count) renderer.render()

6.6.5 Data Structures

The renderer module uses several data structures to represent 3D mesh information:

Vertices: 3D positions of mesh vertices **Colors**: RGB color values for each vertex **Normals**: Surface normal vectors for lighting **UVs**: Texture coordinates for texture mapping **Faces**: Triangle indices defining mesh topology

Example: .. code-block:: python

Vertex data (N, 3) array vertices = np.array([

[0, 0, 0], # Vertex 0 [1, 0, 0], # Vertex 1 [0, 1, 0] # Vertex 2

], dtype=np.float32)

Color data (N, 3) array colors = np.array([

[1, 0, 0], # Red [0, 1, 0], # Green [0, 0, 1] # Blue

], dtype=np.float32)

Face data (M, 3) array faces = np.array([

[0, 1, 2] # Triangle connecting vertices 0, 1, 2

], dtype=np.uint32)

6.6.6 Rendering Pipeline

The PicoGL rendering pipeline follows these steps:

1. **Data Preparation**: Create MeshData with vertices, colors, normals, etc.
2. **Context Setup**: Initialize GLContext with shaders and OpenGL state
3. **Renderer Creation**: Create appropriate renderer (ObjectRenderer, TextureRenderer, etc.)
4. **Initialization**: Call renderer.initialize() to set up OpenGL resources
5. **Rendering**: Call renderer.render() to draw the mesh

Example: .. code-block:: python

1. Prepare data data = MeshData.from_raw(vertices=vertices, colors=colors)

2. Setup context context = GLContext() context.create_shader_program("vertex.glsl", "fragment.glsl")

3. Create renderer renderer = ObjectRenderer(context=context, data=data)

4. Initialize renderer.initialize_shaders() renderer.initialize()

5. Render (in main loop) renderer.render()

6.6.7 Error Handling

PicoGL renderers include comprehensive error handling:

OpenGL Context Errors: Fallback to legacy rendering **Shader Compilation Errors**: Use fallback shaders **Texture Loading Errors**: Skip texture rendering **Mesh Upload Errors**: Use immediate mode rendering

Example: .. code-block:: python

try:
renderer.initialize()

except OpenGLError as e:
print(f'OpenGL error: {e}") # Fallback to legacy rendering

except Exception as e:
print(f'Unexpected error: {e}") # Handle gracefully

6.6.8 Performance Considerations

Modern Renderers (ObjectRenderer, TextureRenderer): * Best performance with modern OpenGL * Requires OpenGL 3.3+ support * Uses VAO/VBO for efficient rendering * Supports advanced features

Legacy Renderers (LegacyGLMesh): * Compatible with older OpenGL versions * Uses immediate mode or legacy VBOs * Good performance on older hardware * Limited feature set

Minimal Renderers (Immediate mode): * Maximum compatibility * Uses glBegin/glEnd for rendering * Lower performance but works everywhere * No advanced features

6.6.9 Best Practices

1. **Choose the right renderer** for your target platform
2. **Use MeshData.from_raw()** for easy data creation
3. **Initialize renderers once** and reuse them
4. **Handle errors gracefully** with fallbacks

5. **Test on multiple platforms** for compatibility

Example: .. code-block:: python

Choose renderer based on OpenGL support if has_modern_opengl():

renderer = ObjectRenderer(context, data)

elif has_legacy_opengl():
renderer = LegacyGLMesh(vertices, faces, colors)

else:
Use immediate mode fallback renderer = ImmediateModeRenderer(vertices, colors)

6.7 Buffers API

The buffers module provides OpenGL buffer management functionality for PicoGL, including vertex buffers, element buffers, and vertex array objects.

6.7.1 Core Classes

VertexArrayObject

`class picogl.backend.modern.core.vertex.array.object.`**`VertexArrayObject`**`(`*handle: int = None*`)`

Bases: `VertexBase`

OpenGL Vertex Array Objects (VAO) class

`__init__``(`*handle: int = None*`)`

VertexArrayObject

Parameters
`handle` – int Handle (ID) of the OpenGL Vertex Array Object (VAO).

`bind``()`

Bind the VAO for use in rendering.

`unbind``()`

Unbind the VAO by binding to zero.

`delete``()`

Delete the VAO from GPU memory.

`set_layout``(`*layout: LayoutDescriptor*`)` → None

Parameters
`layout` – LayoutDescriptor: The layout descriptor to define the vertex attribute format.

Raises
None

Sets the layout for the rendering setup by binding the buffers and configuring the attributes. The state is stored in the Vertex Array Object (VAO). This method assumes a single Vertex Buffer Object (VBO) holds all position data but can be adapted as required. Handles optional usage of Normal Buffer Object (NBO) and Element Buffer Object (EBO) if present.

`add_vbo_object``(`*name: str, vbo: LegacyVBO*`)` → LegacyVBO

Register a VBO by semantic name or shorthand alias.

`get_vbo_object``(`*name: str*`)` → LegacyVBO

Retrieve a VBO by its semantic or shorthand name.

add_vbo(*index: int, data: ndarray, size: int, dtype: int = OpenGL.raw.GL._types.GL_FLOAT, name: str = None, handle: int = None*) → ModernVBO

Add a Vertex Buffer Object (VBO) to the VAO and set its attributes.

Parameters

- **handle**
- **index** – VAO attribute index
- **data** – Vertex data
- **size** – Size per vertex (e.g., 3 for vec3)
- **dtype** – OpenGL data type (e.g., GL_FLOAT)
- **name** – Optional semantic name (e.g., "position", "color")

Returns

OpenGL buffer handle (GLuint)

delete_buffers()

Returns

None

add_attribute(*index: int, vbo: int, size: int = 3, dtype: int = OpenGL.raw.GL._types.GL_FLOAT, normalized: bool = False, stride: int = 0, offset: int = 0*)

Parameters

- **index** – int Index of the vertex attribute.
- **vbo** – int Vertex Buffer Object (VBO) associated with this attribute.
- **size** – int Size of the vertex attribute (e.g., 3 for a 3D vector).
- **dtype** – int Data type of the vertex attribute (default is GL_FLOAT).
- **normalized** – bool Whether the data is normalized (default is False).
- **stride** – int Byte offset between consecutive vertex attributes (default is 0).
- **offset** – int Byte offset to the first component of the

vertex attribute (default is 0). Add a vertex attribute to the VAO.

add_ebo(*data: ndarray*) → *ModernEBO*

Parameters

data – np.ndarray

Returns

int

set_ebo(*ebo: int*) → int

add_ebo

Parameters

ebo – int

Returns

int

property index_count: str | int | None

Return the number of indices in the EBO.

Returns

int

draw(*index_count: int = None, dtype: int = OpenGL.raw.GL._types.GL_UNSIGNED_INT, mode: int = OpenGL.raw.GL.VERSION.GL_1_0.GL_POINTS, pointer: int = c_void_p(None)*)

Parameters

- **pointer** – ctypes.c_void_p(0)
- **dtype** – GL_UNSIGNED_INT
- **index_count** – int Number of vertices to draw.
- **mode** – int e.g. GL_POINT

Returns
None

The `VertexArrayObject` class manages OpenGL Vertex Array Objects (VAOs) for modern OpenGL rendering.

Example: .. code-block:: python

from picogl.backend.modern.core.vertex.array.object import VertexArrayObject

Create VAO vao = VertexArrayObject()

Add vertex buffer vao.add_vbo(index=0, data=vertices, size=3)

Add color buffer vao.add_vbo(index=1, data=colors, size=3)

Add element buffer vao.add_ebo(data=indices)

Draw vao.draw(mode=GL_TRIANGLES, index_count=len(indices))

VertexBufferGroup

The `VertexBufferGroup` class provides legacy OpenGL buffer management for systems without VAO support.

Example: .. code-block:: python

from picogl.buffers.vertex.legacy import VertexBufferGroup

Create vertex buffer group vbg = VertexBufferGroup()

Add vertex buffer vbg.add_vbo("position", vertices, 3)

Add color buffer vbg.add_vbo("color", colors, 3)

Add element buffer vbg.add_ebo(indices)

Bind and draw vbg.bind() vbg.draw(mode=GL_TRIANGLES)

6.7.2 Buffer Classes

Modern Buffers

ModernVBO

ModernEBO

class picogl.backend.modern.core.vertex.buffer.element.**ModernEBO**(*handle: int | None = None, data: ndarray | None = None, size: int = 3, target: int = OpenGL.raw.GL.VERSION.GL_1_5.GL_ELE*

Bases: `VertexBuffer`

OpenGL element data buffer (also known as an index buffer)

__init__(*handle: int | None = None, data: ndarray | None = None, size: int = 3, target: int = OpenGL.raw.GL.VERSION.GL_1_5.GL_ELEMENT_ARRAY_BUFFER*)

set_element_attributes(*data: ndarray*, *size: int*, *dtype: int = OpenGL.raw.GL.VERSION.GL_1_5.GL_STATIC_DRAW*)

Parameters

- **data** – np.ndarray
- **size** – int
- **dtype** – int

Returns
None

configure()

Returns
None

Legacy Buffers

LegacyVBO

LegacyPositionVBO

LegacyColorVBO

LegacyNormalVBO

LegacyEBO

6.7.3 Base Classes

VertexBase

class picogl.buffers.base.**VertexBase**(*handle: int = None*)

Bases: `AbstractVertexGroup`

Generic OpenGL object interface with binding lifecycle.

Provides handle + context manager, leaves binding to subclasses.

__init__(*handle: int = None*)

bind()
Bind the underlying VAO/state for rendering.

unbind()
Optionally unbind the VAO/state.

delete()
Release resources (VAO or equivalent).

attach_buffers(*nbo=None*, *cbo=None*, *vbo=None*, *ebo=None*) → None
Attach the buffers that the VAO/group should coordinate.

set_layout(*layout: LayoutDescriptor*) → None
Define the attribute layout for this VAO/group.

The `VertexBase` class provides the base functionality for all vertex buffer classes.

VertexBuffer

The `VertexBuffer` class provides the base functionality for vertex buffer objects.

6.7.4 Data Structures

LayoutDescriptor

class `picogl.buffers.factory.layout.`**`LayoutDescriptor`**(*attributes: List[*picogl.buffers.attributes.AttributeSpec*]*)

Bases: `object`

attributes: `List[AttributeSpec]`

`__init__`(*attributes: List[*AttributeSpec*]*) → None

The `LayoutDescriptor` class describes the layout of vertex attributes in a buffer.

Example: .. code-block:: python

from picogl.buffers.factory.layout import create_layout

Create layout descriptor layout = create_layout([

AttributeSpec(name=”position”, size=3, type=GL_FLOAT), AttributeSpec(name=”color”, size=3, type=GL_FLOAT), AttributeSpec(name=”normal”, size=3, type=GL_FLOAT)

])

AttributeSpec

class `picogl.buffers.attributes.`**`AttributeSpec`**(*name: str, index: int, size: int, type: int, normalized: bool, stride: int, offset: int*)

Bases: `object`

name: `str`

index: `int`

size: `int`

type: `int`

normalized: `bool`

stride: `int`

offset: `int`

`__init__`(*name: str, index: int, size: int, type: int, normalized: bool, stride: int, offset: int*) → None

The `AttributeSpec` class describes a single vertex attribute.

Example: .. code-block:: python

from picogl.buffers.attributes import AttributeSpec

Create attribute specification position_attr = AttributeSpec(

name=”position”, size=3, type=GL_FLOAT, normalized=False, stride=0, offset=0

)

6.7.5 Buffer Management

Creating Buffers

Modern OpenGL (VAO/VBO): .. code-block:: python from picogl.backend.modern.core.vertex.array.object import VertexArrayObject

Create VAO vao = VertexArrayObject()

Add vertex buffer vao.add_vbo(index=0, data=vertices, size=3)

Add color buffer vao.add_vbo(index=1, data=colors, size=3)

Add element buffer vao.add_ebo(data=indices)

Legacy OpenGL (VBO only): .. code-block:: python

from picogl.buffers.vertex.legacy import VertexBufferGroup

Create vertex buffer group vbg = VertexBufferGroup()

Add vertex buffer vbg.add_vbo("position", vertices, 3)

Add color buffer vbg.add_vbo("color", colors, 3)

Add element buffer vbg.add_ebo(indices)

Binding Buffers

Modern OpenGL: .. code-block:: python

Bind VAO (automatically binds all VBOs) with vao:

Draw calls here vao.draw(mode=GL_TRIANGLES, index_count=len(indices))

Legacy OpenGL: .. code-block:: python

Bind vertex buffer group vbg.bind()

Draw calls here vbg.draw(mode=GL_TRIANGLES)

Unbind vbg.unbind()

6.7.6 Buffer Types

Vertex Buffers

Vertex buffers store vertex data such as positions, colors, normals, and texture coordinates.

Position Buffer: .. code-block:: python

3D positions (N, 3) array vertices = np.array([

[0, 0, 0], # Vertex 0 [1, 0, 0], # Vertex 1 [0, 1, 0] # Vertex 2

], dtype=np.float32)

Color Buffer: .. code-block:: python

RGB colors (N, 3) array colors = np.array([

[1, 0, 0], # Red [0, 1, 0], # Green [0, 0, 1] # Blue

], dtype=np.float32)

Normal Buffer: .. code-block:: python

Surface normals (N, 3) array normals = np.array([

[0, 0, 1], # Normal 0 [0, 0, 1], # Normal 1 [0, 0, 1] # Normal 2

], dtype=np.float32)

UV Buffer: .. code-block:: python

Texture coordinates (N, 2) array uvs = np.array([

[0, 0], # UV 0 [1, 0], # UV 1 [0, 1] # UV 2

], dtype=np.float32)

Element Buffers

Element buffers store indices that define the topology of the mesh.

Element Buffer: .. code-block:: python

Triangle indices (M, 3) array indices = np.array([

[0, 1, 2] # Triangle connecting vertices 0, 1, 2

], dtype=np.uint32)

6.7.7 Buffer Operations

Uploading Data

Modern OpenGL: .. code-block:: python

Upload vertex data vao.add_vbo(index=0, data=vertices, size=3)

Upload color data vao.add_vbo(index=1, data=colors, size=3)

Upload element data vao.add_ebo(data=indices)

Legacy OpenGL: .. code-block:: python

Upload vertex data vbg.add_vbo("position", vertices, 3)

Upload color data vbg.add_vbo("color", colors, 3)

Upload element data vbg.add_ebo(indices)

Drawing

Modern OpenGL: .. code-block:: python

Draw with VAO with vao:

vao.draw(mode=GL_TRIANGLES, index_count=len(indices))

Legacy OpenGL: .. code-block:: python

Draw with vertex buffer group vbg.bind() vbg.draw(mode=GL_TRIANGLES) vbg.unbind()

Cleanup

Modern OpenGL: .. code-block:: python

Cleanup VAO vao.delete()

Legacy OpenGL: .. code-block:: python

Cleanup vertex buffer group vbg.delete()

6.7.8 Error Handling

Buffer operations include comprehensive error handling:

OpenGL Context Errors: Check for valid OpenGL context **Buffer Creation Errors**: Handle OpenGL buffer creation failures **Data Upload Errors**: Validate data before uploading **Drawing Errors**: Handle OpenGL drawing failures

Example: .. code-block:: python

try:
vao.add_vbo(index=0, data=vertices, size=3)

except OpenGLError as e:
print(f'OpenGL error: {e}") # Handle gracefully

except ValueError as e:
print(f"Data error: {e}") # Validate data

except Exception as e:
print(f"Unexpected error: {e}") # Handle gracefully

6.7.9 Performance Considerations

Modern Buffers (VAO/VBO): * Best performance with modern OpenGL * Efficient GPU memory usage * Fast drawing operations * Requires OpenGL 3.0+ support

Legacy Buffers (VBO only): * Compatible with older OpenGL versions * Good performance on older hardware * Uses client state management * Limited feature set

Immediate Mode: * Maximum compatibility * Uses glBegin/glEnd for rendering * Lower performance but works everywhere * No buffer management

6.7.10 Best Practices

1. **Use appropriate buffer type** for your target platform
2. **Upload data once** and reuse buffers
3. **Use VAOs** for modern OpenGL when possible
4. **Handle errors gracefully** with fallbacks
5. **Clean up resources** when done

Example: .. code-block:: python

Choose buffer type based on OpenGL support if has_modern_opengl():

vao = VertexArrayObject() vao.add_vbo(index=0, data=vertices, size=3)

elif has_legacy_opengl():
vbg = VertexBufferGroup() vbg.add_vbo("position", vertices, 3)

else:
Use immediate mode fallback pass

6.8 Shaders API

The shaders module provides OpenGL shader management functionality for PicoGL, including shader compilation, program creation, and uniform management.

6.8.1 Core Classes

ShaderProgram

class picogl.backend.modern.core.shader.program.**ShaderProgram**(*shader_name: str = None, vertex_source_file: str = None, fragment_source_file: str = None, glsl_dir: str | Path | None = None*)

Bases: `object`

OpenGL Shader program manager for vertex and fragment shaders.

__init__(*shader_name: str = None, vertex_source_file: str = None, fragment_source_file: str = None, glsl_dir: str | Path | None = None*)

constructor

program_id()

init_shader_from_glsl_files(*vertex_source_file: str*, *fragment_source_file: str*, *glsl_dir: str | Path | None = None*) → None

Parameters

- **glsl_dir** – directory containing vertex shaders
- **vertex_source_file** – list of paths to vertex shaders
- **fragment_source_file** – list of paths to fragment shaders

Returns
None

init_shader_from_glsl(*vertex_source: str*, *fragment_source: str*) → None

Parameters

- **vertex_source** – list of paths to vertex shaders
- **fragment_source** – list of paths to fragment shaders

Returns
None

init_shader(*vertex_source: str*, *fragment_source: str*)

Parameters

- **vertex_source** – list of paths to vertex shaders
- **fragment_source** – list of paths to fragment shaders

Returns
None

Create, compile, and link shaders into a program.

uniform(*name: str*, *value*)

Parameters

- **name** – str - uniform name
- **value** – value to set (float, int, vec2, vec3, vec4, mat4, or np.ndarray)

Returns
self - for chaining

Set uniform value (auto-detect type)

create_shader_program()

link_shader_program()

get_uniform_location(*uniform_name*)

begin()

end()

bind()
begin

unbind()

release()

delete()

The `ShaderProgram` class manages OpenGL shader programs, including vertex and fragment shaders.

Example: .. code-block:: python

from picogl.backend.modern.core.shader.program import ShaderProgram

Create shader program shader = ShaderProgram(

vertex_source_file="vertex.glsl", fragment_source_file="fragment.glsl"

)

Use shader with shader:

shader.uniform("mvp_matrix", mvp_matrix) shader.uniform("color", [1.0, 0.0, 0.0]) # Draw calls here

ShaderManager

The `ShaderManager` class provides centralized shader management for multiple shader programs.

Example: .. code-block:: python

from picogl.shaders.manager import ShaderManager

Create shader manager manager = ShaderManager(shader_directory="glsl/")

Load shaders manager.load_shader(ShaderType.BASIC, 1) manager.load_shader(ShaderType.TEXTURE, 2)

Use shader shader = manager.get(ShaderType.BASIC) with shader:

Draw calls here

6.8.2 Shader Types

ShaderType

class picogl.shaders.type.**ShaderType**(**values*)

Bases: `str`, `Enum`

AXIS = 'axis'

ATOMS = 'atoms'

BONDS = 'bonds'

DEFAULT = 'default'

CALPHAS = 'calphas'

RIBBONS = 'ribbons'

ISOSURFACE = 'isosurface'

get_name()

get_display_name()

vertex_path()

fragment_path()

The `ShaderType` enum defines the available shader types in PicoGL.

Available Types: * `BASIC`: Basic color shader * `TEXTURE`: Texture mapping shader * `LIGHTING`: Lighting and shading shader * `NORMAL_MAPPING`: Normal mapping shader * `TRANSPARENT`: Transparent rendering shader * `TEXT`: Text rendering shader

Example: .. code-block:: python

from picogl.shaders.type import ShaderType

Use shader type shader_type = ShaderType.BASIC shader = manager.get(shader_type)

6.8.3 Shader Compilation

Compile Shaders

`picogl.shaders.compile.`**`compile_shaders`**(*vertex_src: str*, *fragment_src: str*, *shader_name: str* | *None*) → *ShaderProgram* | None

Compiles and links a vertex + fragment shader_manager.current_shader_program shader_program using Qt's OpenGL API.

Parameters

- **shader_name** – str
- **vertex_src** – Vertex shader_manager.current_shader_program GLSL code.
- **fragment_src** – Fragment shader_manager.current_shader_program GLSL code.

Returns
Linked PicoGLShader or None on failure.

Compiles vertex and fragment shader source code into OpenGL shaders.

Example: .. code-block:: python

from picogl.shaders.compile import compile_shaders

Compile shaders vertex_source = """ #version 330 core layout(location = 0) in vec3 position; uniform mat4 mvp_matrix; void main() {

gl_Position = mvp_matrix * vec4(position, 1.0);

fragment_source = """ #version 330 core out vec4 color; uniform vec3 color_uniform; void main() {

color = vec4(color_uniform, 1.0);

shader = compile_shaders(vertex_source, fragment_source, "basic")

Load Shaders

`picogl.shaders.load.`**`load_fragment_and_vertex_for_shader_type`**(*shader_type_value: str*, *shader_directory: str*) → tuple[str, str]

Parameters

- **shader_directory** – str
- **shader_type_value** – ShaderType

Returns
None

Loads vertex and fragment shader source code from files.

Example: .. code-block:: python

from picogl.shaders.load import load_fragment_and_vertex_for_shader_type

Load shader files vertex_src, fragment_src = load_fragment_and_vertex_for_shader_type(

"basic", "glsl/"

)

Compile shaders shader = compile_shaders(vertex_src, fragment_src, "basic")

6.8.4 Uniform Management

ShaderUniform

`class picogl.shaders.shader_uniform.ShaderUniform`(*name: str, location: int, value: int | float | Sequence[float] | numpy.ndarray | bool | NoneType = None, uniform_type: picogl.shaders.shader_uniform.UniformKind | None = None*)

Bases: `object`

`name: str`

`location: int`

`value: int | float | Sequence[float] | ndarray | bool | None = None`

`uniform_type: UniformKind | None = None`

`static infer_type`(*v: Any*) → UniformKind

`upload`(*gl_module=None*) → None

Upload the current value to the bound GL program using PyOpenGL-like calls. If location < 0 (GL returns -1 when not found/used), this is a no-op.

- gl_module: optional OpenGL.GL-like module. If None, will try to import PyOpenGL (from OpenGL import GL as GL) at call time.

`__init__`(*name: str, location: int, value: int | float | Sequence[float] | ndarray | bool | None = None, uniform_type: UniformKind | None = None*) → None

The `ShaderUniform` class manages OpenGL shader uniforms.

Example: .. code-block:: python

from picogl.shaders.shader_uniform import ShaderUniform

Create uniform uniform = ShaderUniform(shader, "mvp_matrix")

Set uniform value uniform.set(mvp_matrix)

Uniform Types

PicoGL supports various uniform types:

Matrix Uniforms: .. code-block:: python

4x4 matrix shader.uniform("mvp_matrix", mvp_matrix)

3x3 matrix shader.uniform("normal_matrix", normal_matrix)

Vector Uniforms: .. code-block:: python

3D vector shader.uniform("color", [1.0, 0.0, 0.0])

4D vector shader.uniform("position", [0.0, 0.0, 0.0, 1.0])

Scalar Uniforms: .. code-block:: python

Float shader.uniform("time", 0.5)

Integer shader.uniform("texture_sampler", 0)

Boolean Uniforms: .. code-block:: python

Boolean shader.uniform("use_texture", True)

6.8.5 Built-in Shaders

PicoGL includes several built-in shader programs:

Basic Color Shader

Vertex Shader: .. code-block:: glsl

#version 330 core layout(location = 0) in vec3 position; layout(location = 1) in vec3 color;

uniform mat4 mvp_matrix;

out vec3 fragment_color;

void main() {
gl_Position = mvp_matrix * vec4(position, 1.0); fragment_color = color;

}

Fragment Shader: .. code-block:: glsl

#version 330 core in vec3 fragment_color; out vec4 color;

void main() {
color = vec4(fragment_color, 1.0);

}

Texture Shader

Vertex Shader: .. code-block:: glsl

#version 330 core layout(location = 0) in vec3 position; layout(location = 1) in vec2 uv;

uniform mat4 mvp_matrix;

out vec2 fragment_uv;

void main() {
gl_Position = mvp_matrix * vec4(position, 1.0); fragment_uv = uv;

}

Fragment Shader: .. code-block:: glsl

#version 330 core in vec2 fragment_uv;

uniform sampler2D texture0;

out vec4 color;

void main() {
color = texture(texture0, fragment_uv);

}

Lighting Shader

Vertex Shader: .. code-block:: glsl

#version 330 core layout(location = 0) in vec3 position; layout(location = 1) in vec3 normal;

uniform mat4 mvp_matrix; uniform mat4 model_matrix;

out vec3 frag_position; out vec3 frag_normal;

void main() {
gl_Position = mvp_matrix * vec4(position, 1.0); frag_position = vec3(model_matrix * vec4(position, 1.0)); frag_normal = mat3(transpose(inverse(model_matrix))) * normal;

}

Fragment Shader: .. code-block:: glsl

#version 330 core in vec3 frag_position; in vec3 frag_normal;

uniform vec3 light_pos; uniform vec3 view_pos; uniform vec3 object_color;

out vec4 color;

void main() {
vec3 norm = normalize(frag_normal); vec3 light_dir = normalize(light_pos - frag_position);

float diff = max(dot(norm, light_dir), 0.0); vec3 diffuse = diff * object_color;

vec3 ambient = 0.1 * object_color; vec3 result = ambient + diffuse;

color = vec4(result, 1.0);

}

Fallback Shaders

PicoGL includes fallback shaders for systems with limited OpenGL support:

Fallback Vertex Shader: .. code-block:: glsl

#version 120 attribute vec3 in_position; uniform mat4 mvp;

void main() {
gl_Position = mvp * vec4(in_position, 1.0);

}

Fallback Fragment Shader: .. code-block:: glsl

#version 120 void main() {

gl_FragColor = vec4(1.0, 0.0, 1.0, 1.0); // Hot pink

}

6.8.6 Shader Usage

Creating Shader Programs

From Files: .. code-block:: python

from picogl.backend.modern.core.shader.program import ShaderProgram

Create shader from files shader = ShaderProgram(

vertex_source_file="vertex.glsl", fragment_source_file="fragment.glsl", glsl_dir="glsl/"

)

From Source: .. code-block:: python

Create shader from source code shader = ShaderProgram(

vertex_source=vertex_source, fragment_source=fragment_source

)

Using Shaders

Basic Usage: .. code-block:: python

Use shader with shader:

shader.uniform("mvp_matrix", mvp_matrix) shader.uniform("color", [1.0, 0.0, 0.0]) # Draw calls here

Advanced Usage: .. code-block:: python

Bind shader shader.bind()

Set uniforms shader.uniform("mvp_matrix", mvp_matrix) shader.uniform("model_matrix", model_matrix) shader.uniform("view_pos", camera_position)

Draw mesh.draw()

Unbind shader shader.unbind()

6.8.7 Error Handling

Shader operations include comprehensive error handling:

Compilation Errors: Check shader source code **Link Errors**: Check shader program compatibility **Uniform Errors**: Validate uniform names and types **Context Errors**: Check OpenGL context

Example: .. code-block:: python

try:

shader = ShaderProgram(
vertex_source_file="vertex.glsl", fragment_source_file="fragment.glsl"

)

except ShaderCompilationError as e:
print(f"Shader compilation error: {e}") # Check shader source code

except ShaderLinkError as e:
print(f"Shader link error: {e}") # Check shader compatibility

except Exception as e:
print(f"Unexpected error: {e}") # Handle gracefully

6.8.8 Performance Considerations

Modern Shaders (OpenGL 3.3+): * Best performance and features * Full shader pipeline support * Advanced uniform types * Requires modern OpenGL

Legacy Shaders (OpenGL 1.x/2.x): * Compatible with older OpenGL versions * Limited shader features * Uses fixed function pipeline * Good performance on older hardware

Fallback Shaders: * Maximum compatibility * Basic functionality only * Uses immediate mode rendering * Works on any OpenGL system

6.8.9 Best Practices

1. **Use appropriate shader type** for your target platform
2. **Compile shaders once** and reuse them
3. **Handle compilation errors** gracefully
4. **Use fallback shaders** for compatibility
5. **Test on multiple platforms** for compatibility

Example: .. code-block:: python

Choose shader based on OpenGL support if has_modern_opengl():

shader = ShaderProgram(
vertex_source_file="vertex.glsl", fragment_source_file="fragment.glsl"

)

elif has_legacy_opengl():

shader = ShaderProgram(
vertex_source_file="legacy_vertex.glsl", fragment_source_file="legacy_fragment.glsl"

)

else:
Use fallback shader shader = ShaderProgram(

vertex_source=fallback_vertex_source, fragment_source=fallback_fragment_source

)

6.9 UI API

The UI module provides user interface functionality for PicoGL, including window management, input handling, and platform-specific backends.

6.9.1 Core Classes

AbstractGLWindow

`class picogl.ui.abc_window.AbstractGLWindow`(*width: int = 800*, *height: int = 480*, *title: bytes = b'GL Window'*)

Bases: `ABC`

A strict ABC base class for a GLUT/OpenGL window.

Subclasses must implement:

- initializeGL
- paintGL
- resizeGL
- on_keyboard
- on_special_key
- on_mouse
- on_mousemove

`__init__`(*width: int = 800*, *height: int = 480*, *title: bytes = b'GL Window'*)

`abstractmethod initializeGL()` → None

Set up OpenGL state. Must be implemented by subclass.

`abstractmethod paintGL()` → None

Render the scene. Must be implemented by subclass.

`abstractmethod resizeGL`(*width: int*, *height: int*) → None

Handle window resize. Must be implemented by subclass.

`display()` → None

Default display path calls paintGL, then swaps buffers

idle() → None
: Optional idle hook (override if needed).

abstractmethod **on_keyboard**(*key*, *x*, *y*) → None
: Handle ASCII keyboard input. Must be implemented by subclass.

abstractmethod **on_special_key**(*key*, *x*, *y*) → None
: Handle special keys (arrows, function keys). Must be implemented by subclass.

abstractmethod **mousePressEvent**(**args*, ***kwargs*) → None
: Handle mouse button events. Must be implemented by subclass.

abstractmethod **mouseMoveEvent**(**args*, ***kwargs*) → None
: Handle mouse movement events. Must be implemented by subclass.

run() → None

The `AbstractGLWindow` class defines the interface for all PicoGL window implementations.

Example: .. code-block:: python

from picogl.ui.abc_window import AbstractGLWindow

class MyWindow(AbstractGLWindow):

def __init__(self):
super().__init__() self.width = 800 self.height = 600

def initializeGL(self):
Initialize OpenGL pass

def paintGL(self):
Render scene pass

def resizeGL(self, width, height):
Handle resize pass

6.9.2 Window Backends

GLUT Backend

GLWindow

class picogl.ui.backend.glut.window.gl.**GLWindow**(*title: str = 'window'*, **args*, ***kwargs*)
: Bases: *AbstractGLWindow*

__init__(*title: str = 'window'*, **args*, ***kwargs*)

init_glut()

initializeGL()
: initialize_gl

paintGL()

update()
: draw

display()

idle()

resizeGL(*width: int*, *height: int*)
: resize

on_keyboard(*key*, *x*, *y*)

on_special_key(*key*, *x*, *y*)

mousePressEvent(**args*, ***kwargs*)

on_mouse

mouseMoveEvent(**args*, ***kwargs*)

on_mousemove

run()

The `GLWindow` class provides a basic GLUT-based window implementation.

Example: .. code-block:: python

from picogl.ui.backend.glut.window.gl import GLWindow

Create GLUT window window = GLWindow(title="My Window") window.initializeGL() window.run()

GlutRendererWindow

class picogl.ui.backend.glut.window.glut.**GlutRendererWindow**(*width*, *height*, *title: str = None*, *context: GLContext = None*, **args*, ***kwargs*)

Bases: *GLWindow*

Glut Rendered Window

__init__(*width*, *height*, *title: str = None*, *context: GLContext = None*, **args*, ***kwargs*)

initializeGL()

Initial OpenGL configuration.

calculate_mvp_matrix(*width: int = 1920*, *height: int = 1080*)

resizeGL(*width*, *height*)

paintGL()

update_mvp()

Base perspective matrix from your existing method

mousePressEvent(*button*, *state*, *x*, *y*)

mouseMoveEvent(*x*, *y*)

wheelEvent(*wheel=0*, *direction=0*, *x=0*, *y=0*)

Mouse wheel zoom: adjusts distance if far, FOV if close. Positive direction -> zoom in, Negative -> zoom out.

get_size()

The `GlutRendererWindow` class provides a GLUT window with integrated rendering capabilities.

Example: .. code-block:: python

from picogl.ui.backend.glut.window.glut import GlutRendererWindow from picogl.renderer import GLContext, MeshData

Create renderer window context = GLContext() data = MeshData.from_raw(vertices=vertices, colors=colors)

window = GlutRendererWindow(

width=800, height=600, title="Renderer Window", context=context, data=data

)

window.initializeGL() window.run()

RenderWindow

class picogl.ui.backend.glut.window.object.**RenderWindow**(*width: int = 800*, *height: int = 600*, *title: str = 'RenderWindow'*, *data: MeshData | None = None*, *base_dir: str | Path | None = None*, *glsl_dir: str | Path | None = None*, *use_texture: bool = False*, *texture_file: str | None = None*, *resource_subdir: str = 'tu02'*, **args*, ***kwargs*)

Bases: `GlutRendererWindow`

Unified render window supporting textured or untextured rendering.

__init__(*width: int = 800*, *height: int = 600*, *title: str = 'RenderWindow'*, *data: MeshData | None = None*, *base_dir: str | Path | None = None*, *glsl_dir: str | Path | None = None*, *use_texture: bool = False*, *texture_file: str | None = None*, *resource_subdir: str = 'tu02'*, **args*, ***kwargs*)

initialize()

The `RenderWindow` class provides a unified render window supporting both textured and untextured rendering.

Example: .. code-block:: python

from picogl.ui.backend.glut.window.object import RenderWindow from picogl.renderer import MeshData

Create render window data = MeshData.from_raw(vertices=vertices, colors=colors)

window = RenderWindow(
width=800, height=600, title="Render Window", data=data, use_texture=False

)

window.initialize() window.run()

TextureWindow

class picogl.ui.backend.glut.window.texture.**TextureWindow**(*width: int*, *height: int*, *title: str*, *data: MeshData*, *base_dir: str | Path*, *glsl_dir: str | Path*, *use_texture: bool*, **args*, ***kwargs*)

Bases: `GlutRendererWindow`

file with stubs for actions

__init__(*width: int*, *height: int*, *title: str*, *data: MeshData*, *base_dir: str | Path*, *glsl_dir: str | Path*, *use_texture: bool*, **args*, ***kwargs*)

The `TextureWindow` class provides a window specialized for texture rendering.

Example: .. code-block:: python

from picogl.ui.backend.glut.window.texture import TextureWindow from picogl.renderer import MeshData

Create texture window data = MeshData.from_raw(vertices=vertices, uvs=uvs)

window = TextureWindow(
width=800, height=600, title="Texture Window", data=data, base_dir="resources/", use_texture=True

)

window.initialize() window.run()

Qt Backend

GLBase

The `GLBase` class provides a Qt-based OpenGL widget implementation.

Example: .. code-block:: python

from picogl.ui.backend.qt.base import GLBase from PyQt5.QtWidgets import QApplication, QMain-Window

class MyGLWidget(GLBase):

def __init__(self):
super().__init__()

def initializeGL(self):
Initialize OpenGL pass

def paintGL(self):
Render scene pass

Create Qt application app = QApplication([]) window = QMainWindow() widget = MyGLWidget() window.setCentralWidget(widget) window.show() app.exec_()

6.9.3 Input Handling

Mouse Input

Mouse Press Events: .. code-block:: python

def mousePressEvent(self, button, state, x, y):

if button == GLUT_LEFT_BUTTON:

if state == GLUT_DOWN:
self.last_mouse_x = x self.last_mouse_y = y

else:
self.last_mouse_x = None self.last_mouse_y = None

Mouse Motion Events: .. code-block:: python

def motion(self, x, y):

if self.last_mouse_x is not None and self.last_mouse_y is not None:
dx = x - self.last_mouse_x dy = y - self.last_mouse_y

self.rotation_y += dx * 0.5 self.rotation_x += dy * 0.5

self.last_mouse_x = x self.last_mouse_y = y glutPostRedisplay()

Mouse Wheel Events: .. code-block:: python

def mouse(self, button, state, x, y):

if button == 3: # Mouse wheel up
self.zoom_distance = max(1.0, self.zoom_distance - 0.5)

elif button == 4: # Mouse wheel down
self.zoom_distance = min(20.0, self.zoom_distance + 0.5)

glutPostRedisplay()

Keyboard Input

Keyboard Events: .. code-block:: python

> **def keyboard(self, key, x, y):**
>
> > **if key == b'x1b': # ESC key**
> > sys.exit(0)
> >
> > **elif key == b'r': # Reset rotation**
> > self.rotation_x = 0.0 self.rotation_y = 0.0
> >
> > **elif key == b'w': # Toggle wireframe**
> > self.wireframe_mode = not self.wireframe_mode
> >
> > glutPostRedisplay()

Special Keys: .. code-block:: python

> **def on_special_key(self, key, x, y):**
>
> > **if key == GLUT_KEY_UP:**
> > self.rotation_x += 5.0
> >
> > **elif key == GLUT_KEY_DOWN:**
> > self.rotation_x -= 5.0
> >
> > **elif key == GLUT_KEY_LEFT:**
> > self.rotation_y += 5.0
> >
> > **elif key == GLUT_KEY_RIGHT:**
> > self.rotation_y -= 5.0
> >
> > glutPostRedisplay()

6.9.4 Window Management

Creating Windows

Basic Window: .. code-block:: python

> from picogl.ui.backend.glut.window.gl import GLWindow
>
> # Create basic window window = GLWindow(title="My Window") window.initializeGL() window.run()

Renderer Window: .. code-block:: python

> from picogl.ui.backend.glut.window.object import RenderWindow from picogl.renderer import MeshData
>
> # Create renderer window data = MeshData.from_raw(vertices=vertices, colors=colors)
>
> **window = RenderWindow(**
> width=800, height=600, title="Renderer Window", data=data
>
>)
>
> window.initialize() window.run()

Window Properties

Size and Position: .. code-block:: python

> # Set window size window.width = 800 window.height = 600
>
> # Set window position window.x = 100 window.y = 100

Title and Icon: .. code-block:: python

Set window title window.title = "My PicoGL Application"

Set window icon (if supported) window.icon = "icon.png"

Fullscreen Mode: .. code-block:: python

Toggle fullscreen window.toggle_fullscreen()

Check fullscreen state if window.is_fullscreen():

print("Window is in fullscreen mode")

6.9.5 Rendering Loop

Display Function

Basic Display: .. code-block:: python

def display(self):
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT) glLoadIdentity()

Set up camera gluLookAt(0, 0, 5, 0, 0, 0, 0, 1, 0)

Apply transformations glRotatef(self.rotation_x, 1, 0, 0) glRotatef(self.rotation_y, 0, 1, 0)

Draw scene self.draw_scene()

glutSwapBuffers()

Advanced Display: .. code-block:: python

def display(self):
Clear buffers glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT) glLoadIdentity()

Set up camera self.setup_camera()

Apply transformations self.apply_transformations()

Draw scene self.draw_scene()

Draw UI elements self.draw_ui()

Swap buffers glutSwapBuffers()

Resize Handling

Basic Resize: .. code-block:: python

def resizeGL(self, width, height):
self.width = width self.height = height glViewport(0, 0, width, height) glMatrixMode(GL_PROJECTION) glLoadIdentity() gluPerspective(45.0, float(width) / float(height), 0.1, 100.0) glMatrixMode(GL_MODELVIEW)

Advanced Resize: .. code-block:: python

def resizeGL(self, width, height):
self.width = width self.height = height

Update viewport glViewport(0, 0, width, height)

Update projection matrix self.update_projection_matrix(width, height)

Update UI layout self.update_ui_layout(width, height)

Notify renderers for renderer in self.renderers:

renderer.resize(width, height)

6.9.6 Event Handling

Event Loop

Basic Event Loop: .. code-block:: python

> **def run(self):**
> glutMainLoop()

Custom Event Loop: .. code-block:: python

> **def run(self):**
>
> **while not self.should_exit:**
> self.handle_events() self.update() self.render() self.sleep(16) # 60 FPS

Idle Function: .. code-block:: python

> **def idle(self):**
>
> **if self.auto_rotate:**
> self.rotation_y += 0.5 glutPostRedisplay()

Timer Events

Basic Timer: .. code-block:: python

> **def timer(self, value):**
> # Update animation self.update_animation()
>
> # Redraw glutPostRedisplay()
>
> # Set next timer glutTimerFunc(16, self.timer, 0) # 60 FPS

Advanced Timer: .. code-block:: python

> **def timer(self, value):**
> # Update physics self.update_physics()
>
> # Update animation self.update_animation()
>
> # Update UI self.update_ui()
>
> # Redraw glutPostRedisplay()
>
> # Set next timer glutTimerFunc(16, self.timer, 0)

6.9.7 Platform-Specific Features

macOS

Display Environment: .. code-block:: python

> import os
>
> # Check for display if os.environ.get('DISPLAY') is None:
>
> print("No display available") return
>
> # Run from Terminal.app or iTerm2 window.run()

OpenGL Context: .. code-block:: python

> # Check OpenGL version import OpenGL.GL as GL version = GL.glGetString(GL.GL_VERSION) print(f"OpenGL version: {version}")

Windows

Graphics Drivers: .. code-block:: python

> # Check graphics vendor import OpenGL.GL as GL vendor = GL.glGetString(GL.GL_VENDOR) print(f"Graphics vendor: {vendor}")

DLL Loading: .. code-block:: python

> # Check for required DLLs try:
>
> > import OpenGL.GL as GL
>
> **except ImportError as e:**
> > print(f"DLL loading failed: {e}") print("Install Visual C++ Redistributable")

Linux

Display Server: .. code-block:: python

> import os
>
> # Check display display = os.environ.get('DISPLAY') if display:
>
> > print(f"Using display: {display}")
>
> **else:**
> > print("No display available")

OpenGL Libraries: .. code-block:: python

> # Check OpenGL libraries try:
>
> > import OpenGL.GL as GL
>
> **except ImportError as e:**
> > print(f"OpenGL libraries not found: {e}") print("Install mesa libraries")

6.9.8 Error Handling

Window Creation Errors

Display Errors: .. code-block:: python

> **try:**
> > window = GLWindow(title="My Window")
>
> **except DisplayError as e:**
> > print(f"Display error: {e}") # Handle gracefully
>
> **except Exception as e:**
> > print(f"Unexpected error: {e}") # Handle gracefully

OpenGL Context Errors: .. code-block:: python

> **try:**
> > window.initializeGL()
>
> **except OpenGLError as e:**
> > print(f"OpenGL error: {e}") # Use fallback rendering
>
> **except Exception as e:**
> > print(f"Unexpected error: {e}") # Handle gracefully

Input Errors

Mouse Input Errors: .. code-block:: python

> **def mousePressEvent(self, button, state, x, y):**

try:
Handle mouse input pass

except Exception as e:
print(f"Mouse input error: {e}") # Handle gracefully

Keyboard Input Errors: .. code-block:: python

def keyboard(self, key, x, y):

try:
Handle keyboard input pass

except Exception as e:
print(f"Keyboard input error: {e}") # Handle gracefully

6.9.9 Performance Considerations

Frame Rate: .. code-block:: python

Target 60 FPS glutTimerFunc(16, self.timer, 0) # 16ms = 60 FPS

Target 30 FPS glutTimerFunc(33, self.timer, 0) # 33ms = 30 FPS

Double Buffering: .. code-block:: python

Enable double buffering glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE | GLUT_DEPTH)

Swap buffers glutSwapBuffers()

VSync: .. code-block:: python

Enable VSync (if supported) import OpenGL.GL as GL GL.glEnable(GL.GL_VSYNC)

6.9.10 Best Practices

1. **Use appropriate window type** for your needs
2. **Handle input events** gracefully
3. **Implement proper resize handling**
4. **Use double buffering** for smooth rendering
5. **Test on multiple platforms** for compatibility

Example: .. code-block:: python

Choose window type based on needs if needs_texture_rendering:

window = TextureWindow(data=data, use_texture=True)

elif needs_basic_rendering:
window = RenderWindow(data=data)

else:
window = GLWindow(title="Basic Window")

Handle errors gracefully try:

window.initialize() window.run()

except Exception as e:
print(f"Window error: {e}") # Handle gracefully

6.10 Utils API

The utils module provides utility functionality for PicoGL, including data loading, texture management, and helper functions.

6.10.1 Core Classes

ObjectLoader

class picogl.utils.loader.object.**ObjectLoader**(*path: str*)

Bases: `object`

Object Loader Class

__init__(*path: str*)

log_properties()

log object properties

to_array_style() → ObjectData

Convert to array-style where each vertex attribute is stored separately

to_single_index_style() → ObjectData

Convert to single-index style where each unique vertex attribute combination is stored once

The `ObjectLoader` class provides functionality for loading 3D object files (OBJ format).

Example: .. code-block:: python

from picogl.utils.loader.object import ObjectLoader

Load OBJ file loader = ObjectLoader("model.obj") data = loader.to_array_style()

Create mesh data from picogl.renderer import MeshData mesh_data = MeshData.from_raw(

vertices=data.vertices, normals=data.normals, colors=data.colors

)

TextureLoader

class picogl.utils.loader.texture.**TextureLoader**(*file_name: str*, *mode: str = 'RGB'*)

Bases: `object`

Loads a 2D texture from a DDS file or a standard image file using PIL. Automatically creates an OpenGL texture ID.

__init__(*file_name: str*, *mode: str = 'RGB'*) → None

load_dds(*file_name: str*) → None

Load a DDS texture from file. Supports DXT1, DXT3, DXT5 compressed textures.

load_by_pil(*file_name: str*, *mode: str*) → None

Load a standard image using PIL and upload as OpenGL texture.

delete() → None

Deletes the OpenGL texture to free GPU memory.

The `TextureLoader` class provides functionality for loading and managing textures.

Example: .. code-block:: python

from picogl.utils.loader.texture import TextureLoader

Load texture loader = TextureLoader("texture.png") texture = loader.load()

Use texture texture.bind() # Draw textured geometry

6.10.2 Helper Functions

GL Initialization

`picogl.utils.gl_init.`**`execute_gl_tasks`**(*task_list: list[tuple[str, Callable]]*)

Execute a sequence of OpenGL-related tasks.

Each task is a tuple `(message, func)`: - `message` (*str* or `None`): If a string, it is logged before running the task.

If `None`, no log message is emitted for that step.

- `func` (*callable*): The function to execute.

Parameters
`task_list` (*list[tuple[str | None, callable]]*) – A list of `(message, callable)` tuples describing the tasks to run.

Raises

- `TypeError` – If `task_list` is not a list or any element is not a 2-tuple.
- `Exception` – Logs and re-raises any exception thrown by a task.

Executes a list of OpenGL initialization tasks.

Example: .. code-block:: python

from picogl.utils.gl_init import execute_gl_tasks, init_gl_list

Execute initialization tasks execute_gl_tasks(init_gl_list)

`picogl.utils.gl_init.`**`init_gl_list`**()

Built-in mutable sequence.

If no argument is given, the constructor creates a new empty list. The argument must be an iterable if specified.

Returns a list of OpenGL initialization tasks.

Example: .. code-block:: python

from picogl.utils.gl_init import init_gl_list

Get initialization tasks tasks = init_gl_list execute_gl_tasks(tasks)

`picogl.utils.gl_init.`**`paint_gl_list`**()

Built-in mutable sequence.

If no argument is given, the constructor creates a new empty list. The argument must be an iterable if specified.

Returns a list of OpenGL painting tasks.

Example: .. code-block:: python

from picogl.utils.gl_init import paint_gl_list

Get painting tasks tasks = paint_gl_list execute_gl_tasks(tasks)

Texture Utilities

`picogl.utils.texture.`**`bind_texture_array`**(*texture_id: int*)

Parameters
`texture_id`

Returns
None

Binds a texture array to the specified texture unit.

Example: .. code-block:: python

from picogl.utils.texture import bind_texture_array

Bind texture to unit 0 bind_texture_array(texture_id, 0)

Normal Generation

Generates surface normals for a mesh.

Example: .. code-block:: python

from picogl.utils.normal import generate_normals

Generate normals normals = generate_normals(vertices, faces)

Reshape Utilities

Handles window resize events.

Example: .. code-block:: python

from picogl.utils.reshape import handle_resize

Handle resize handle_resize(width, height)

6.10.3 Data Loading

OBJ File Loading

Basic Loading: .. code-block:: python

from picogl.utils.loader.object import ObjectLoader

Load OBJ file loader = ObjectLoader("model.obj") data = loader.to_array_style()

Access data vertices = data.vertices normals = data.normals colors = data.colors uvs = data.uvs

Advanced Loading: .. code-block:: python

Load with options loader = ObjectLoader("model.obj") loader.set_options(

normalize=True, generate_normals=True, generate_colors=True

)

data = loader.to_array_style()

Error Handling: .. code-block:: python

try:
loader = ObjectLoader("model.obj") data = loader.to_array_style()

except FileNotFoundError:
print("OBJ file not found")

except ValueError as e:
print(f"Invalid OBJ file: {e}")

except Exception as e:
print(f"Unexpected error: {e}")

Texture Loading

Basic Loading: .. code-block:: python from picogl.utils.loader.texture import TextureLoader

Load texture loader = TextureLoader("texture.png") texture = loader.load()

Use texture texture.bind()

Advanced Loading: .. code-block:: python

Load with options loader = TextureLoader("texture.png") loader.set_options(

flip_vertical=True, generate_mipmaps=True, wrap_mode=GL_REPEAT

)

texture = loader.load()

Error Handling: .. code-block:: python

try:
loader = TextureLoader("texture.png") texture = loader.load()

except FileNotFoundError:
print("Texture file not found")

except ValueError as e:
print(f"Invalid texture file: {e}")

except Exception as e:
print(f"Unexpected error: {e}")

6.10.4 Data Processing

Mesh Data Processing

Vertex Processing: .. code-block:: python

import numpy as np

Normalize vertices vertices = vertices / np.linalg.norm(vertices, axis=1, keepdims=True)

Center vertices center = np.mean(vertices, axis=0) vertices = vertices - center

Scale vertices scale = 1.0 / np.max(np.abs(vertices)) vertices = vertices * scale

Normal Processing: .. code-block:: python

from picogl.utils.normal import generate_normals

Generate normals normals = generate_normals(vertices, faces)

Normalize normals normals = normals / np.linalg.norm(normals, axis=1, keepdims=True)

Color Processing: .. code-block:: python

Generate colors based on normals colors = (normals + 1.0) / 2.0

Generate colors based on vertices colors = (vertices - vertices.min()) / (vertices.max() - vertices.min())

Generate random colors colors = np.random.rand(len(vertices), 3)

Texture Processing

UV Coordinate Processing: .. code-block:: python

Flip V coordinates uvs[:, 1] = 1.0 - uvs[:, 1]

Scale UV coordinates uvs = uvs * 2.0 - 1.0

Wrap UV coordinates uvs = uvs % 1.0

Texture Format Conversion: .. code-block:: python from PIL import Image

Convert to RGBA image = Image.open("texture.png").convert("RGBA")

Convert to RGB image = Image.open("texture.png").convert("RGB")

Resize texture image = image.resize((512, 512))

6.10.5 File Management

Path Handling

Path Operations: .. code-block:: python

from pathlib import Path

Create path base_dir = Path("resources") texture_path = base_dir / "textures" / "texture.png"

Check if path exists if texture_path.exists():

print("Texture file exists")

Get absolute path abs_path = texture_path.absolute()

Resource Management: .. code-block:: python

Find resource files def find_resources(directory, pattern="*.obj"):

return list(Path(directory).glob(pattern))

Load all OBJ files obj_files = find_resources("models", "*.obj") for obj_file in obj_files:

loader = ObjectLoader(str(obj_file)) data = loader.to_array_style()

6.10.6 Error Handling

Loading Errors

File Not Found: .. code-block:: python

try:
loader = ObjectLoader("nonexistent.obj") data = loader.to_array_style()

except FileNotFoundError:
print("File not found, using fallback") # Use fallback data

except Exception as e:
print(f"Unexpected error: {e}")

Invalid File Format: .. code-block:: python

try:
loader = ObjectLoader("invalid.obj") data = loader.to_array_style()

except ValueError as e:
print(f"Invalid file format: {e}") # Handle gracefully

except Exception as e:
print(f"Unexpected error: {e}")

Memory Errors: .. code-block:: python

try:
loader = ObjectLoader("large_model.obj") data = loader.to_array_style()

except MemoryError:
print("File too large, using simplified version") # Use simplified data

except Exception as e:
print(f"Unexpected error: {e}")

6.10.7 Performance Considerations

Lazy Loading: .. code-block:: python

class LazyLoader:

def __init__(self, filename):
self.filename = filename self._data = None

@property def data(self):

if self._data is None:
self._data = self._load_data()

return self._data

def _load_data(self):
loader = ObjectLoader(self.filename) return loader.to_array_style()

Caching: .. code-block:: python

import functools

@functools.lru_cache(maxsize=128) def load_texture(filename):

loader = TextureLoader(filename) return loader.load()

Batch Loading: .. code-block:: python

def load_multiple_objects(filenames):
results = [] for filename in filenames:

try:
loader = ObjectLoader(filename) data = loader.to_array_style() results.append(data)

except Exception as e:
print(f"Failed to load {filename}: {e}") results.append(None)

return results

6.10.8 Best Practices

1. **Use appropriate loader** for your file type
2. **Handle errors gracefully** with fallbacks
3. **Cache loaded data** when possible
4. **Validate data** before using
5. **Use lazy loading** for large files

Example: .. code-block:: python

Choose loader based on file type if filename.endswith('.obj'):

loader = ObjectLoader(filename)

elif filename.endswith(('.png', '.jpg', '.jpeg')):
loader = TextureLoader(filename)

else:
raise ValueError(f"Unsupported file type: {filename}")

Load with error handling try:

data = loader.load()

except Exception as e:
print(f"Failed to load {filename}: {e}") # Use fallback data data = create_fallback_data()

return data

6.11 Development

This section provides information for developers who want to contribute to PicoGL or understand its internal architecture.

6.11.1 Project Structure

PicoGL follows a modular architecture with clear separation of concerns:

```
picogl/
├── backend/           # OpenGL backend implementations
│   ├── modern/        # Modern OpenGL 3.3+ backend
│   └── legacy/        # Legacy OpenGL 1.x/2.x backend
├── buffers/           # Buffer management
│   ├── vertex/        # Vertex buffer implementations
│   └── attributes/    # Attribute specifications
├── renderer/          # Core rendering classes
│   ├── meshdata.py    # Mesh data management
│   ├── object.py      # Object renderer
│   └── texture.py     # Texture renderer
├── shaders/           # Shader management
│   ├── compile.py     # Shader compilation
│   ├── load.py        # Shader loading
│   └── manager.py     # Shader manager
├── ui/                # User interface
│   └── backend/       # UI backends (GLUT, Qt)
├── utils/             # Utility functions
│   ├── loader/        # Data loaders
│   └── texture.py     # Texture utilities
└── tests/             # Unit tests
```

6.11.2 Architecture Overview

PicoGL uses a layered architecture:

Application Layer

- User applications and examples
- High-level API usage

Renderer Layer

- ObjectRenderer, TextureRenderer
- MeshData, GLContext
- High-level rendering abstractions

Backend Layer

- Modern OpenGL 3.3+ implementation
- Legacy OpenGL 1.x/2.x implementation
- Platform-specific optimizations

OpenGL Layer

- Direct OpenGL calls
- Buffer management
- Shader compilation

Platform Layer

- GLUT, Qt backends
- Input handling
- Window management

6.11.3 Design Principles

Compatibility First

- Support for both modern and legacy OpenGL
- Graceful degradation when features unavailable
- Cross-platform compatibility

Educational Focus

- Clean, well-documented code
- Clear separation of concerns
- Easy to understand and modify

Performance Aware

- Efficient rendering pipelines
- Minimal overhead
- Platform-specific optimizations

Extensible Design

- Modular architecture
- Plugin system for backends
- Easy to add new features

6.11.4 Backend System

Modern Backend

The modern backend provides OpenGL 3.3+ functionality:

Features: * Vertex Array Objects (VAOs) * Modern shader pipeline * Advanced uniform types * Efficient buffer management

Key Classes: * `VertexArrayObject`: VAO management * `ModernVBO`: Modern vertex buffers * `ModernEBO`: Modern element buffers * `ShaderProgram`: Shader management

Example: .. code-block:: python

from picogl.backend.modern.core.vertex.array.object import VertexArrayObject

Create VAO vao = VertexArrayObject() vao.add_vbo(index=0, data=vertices, size=3) vao.add_vbo(index=1, data=colors, size=3) vao.add_ebo(data=indices)

Draw with vao:

vao.draw(mode=GL_TRIANGLES, index_count=len(indices))

Legacy Backend

The legacy backend provides OpenGL 1.x/2.x compatibility:

Features: * Immediate mode rendering * Legacy VBOs * Fixed function pipeline * Client state management

Key Classes: * `VertexBufferGroup`: Legacy buffer management * `LegacyVBO`: Legacy vertex buffers * `LegacyEBO`: Legacy element buffers * `LegacyGLMesh`: Legacy mesh rendering

Example: .. code-block:: python

from picogl.buffers.vertex.legacy import VertexBufferGroup

Create vertex buffer group vbg = VertexBufferGroup() vbg.add_vbo("position", vertices, 3) vbg.add_vbo("color", colors, 3) vbg.add_ebo(indices)

Draw vbg.bind() vbg.draw(mode=GL_TRIANGLES) vbg.unbind()

Backend Selection

PicoGL automatically selects the appropriate backend:

```python
def select_backend():
    if has_modern_opengl():
        return ModernBackend()
    elif has_legacy_opengl():
        return LegacyBackend()
    else:
        return ImmediateModeBackend()
```

6.11.5 Testing Framework

Unit Tests

PicoGL includes comprehensive unit tests:

Test Structure: .. code-block:: text

tests/ ├── test_vertex_array_object.py ├── test_vertex_buffer_group.py ├── test_meshdata.py ├── test_glmesh.py ├── test_legacy_glmesh.py ├── test_object_renderer.py └── test_texture_renderer.py

Running Tests: .. code-block:: bash

Run all tests python -m unittest discover tests

Run specific test python -m unittest tests.test_vertex_array_object

Run with coverage python -m pytest tests/ –cov=picogl

Test Features: * Mock OpenGL functions to avoid context requirements * Comprehensive error handling tests * Edge case testing * Performance benchmarks

Example Test: .. code-block:: python

import unittest from unittest.mock import MagicMock, patch from picogl.backend.modern.core.vertex.array.object import VertexArrayObject

class TestVertexArrayObject(unittest.TestCase):

def setUp(self):
Mock OpenGL functions self.gl_patches = [

patch('picogl.backend.modern.core.vertex.array.object.glGenVertexArrays'), patch('picogl.backend.modern.core.vertex.array.object.glBindVertexArray'), # … more patches

] for patch_obj in self.gl_patches:

patch_obj.start()

def tearDown(self):

for patch_obj in self.gl_patches:
patch_obj.stop()

def test_initialization(self):
vao = VertexArrayObject() self.assertIsNotNone(vao)

Integration Tests

Integration tests verify end-to-end functionality:

Test Categories: * Rendering pipeline tests * Cross-platform compatibility tests * Performance regression tests * Memory leak tests

Example: .. code-block:: python

def test_rendering_pipeline():

Create mesh data data = MeshData.from_raw(vertices=vertices, colors=colors)

Create renderer renderer = ObjectRenderer(context=context, data=data)

Test initialization renderer.initialize_shaders() renderer.initialize()

Test rendering renderer.render()

Verify no errors self.assertFalse(has_gl_errors())

6.11.6 Build System

Setup Configuration

PicoGL uses setuptools for packaging:

pyproject.toml: .. code-block:: toml

[build-system] requires = ["setuptools>=45", "wheel"] build-backend = "setuptools.build_meta"

[project] name = "picogl" version = "1.0.0" description = "Lightweight Python OpenGL wrapper" requires-python = ">=3.7" dependencies = [

"numpy>=1.19.0", "PyOpenGL>=3.1.0", "PyOpenGL-accelerate>=3.1.0", "pyglm>=2.0.0", "Pillow>=8.0.0"

]

[project.optional-dependencies] dev = [

"pytest>=6.0.0", "pytest-cov>=2.0.0", "black>=21.0.0", "flake8>=3.8.0", "mypy>=0.800"

] qt = [

"PyQt5>=5.15.0", "PyQt6>=6.0.0"

]

setup.py: .. code-block:: python

from setuptools import setup, find_packages

setup(

name="picogl", version="1.0.0", packages=find_packages(), python_requires=">=3.7", install_requires=[

"numpy>=1.19.0", "PyOpenGL>=3.1.0", "PyOpenGL-accelerate>=3.1.0", "pyglm>=2.0.0", "Pillow>=8.0.0"

], extras_require={

"dev": [

"pytest>=6.0.0", "pytest-cov>=2.0.0", "black>=21.0.0", "flake8>=3.8.0", "mypy>=0.800"

], "qt": [

"PyQt5>=5.15.0", "PyQt6>=6.0.0"

]

}

)

Development Dependencies

Core Dependencies: * Python 3.7+ * NumPy * PyOpenGL * PyGLM * Pillow

Development Dependencies: * pytest: Testing framework * pytest-cov: Coverage testing * black: Code formatting * flake8: Linting * mypy: Type checking

Installation: .. code-block:: bash

    # Install in development mode pip install -e .

    # Install with development dependencies pip install -e .[dev]

    # Install with all optional dependencies pip install -e .[dev,qt]

6.11.7 Code Style

Formatting

PicoGL uses Black for code formatting:

Configuration: .. code-block:: toml

    [tool.black] line-length = 88 target-version = ['py37'] include = '.pyi?$' extend-exclude = '" /(

        .git

      .mypy_cache
      .tox
      .venv
      _build
      buck-out
      build
      dist

Usage: .. code-block:: bash

    # Format all Python files black picogl/

    # Check formatting black –check picogl/

Linting

PicoGL uses flake8 for linting:

Configuration: .. code-block:: ini

    [flake8] max-line-length = 88 extend-ignore = E203, W503 exclude =

      .git, __pycache__, .venv, _build, dist

Usage: .. code-block:: bash

    # Lint all Python files flake8 picogl/

    # Lint specific file flake8 picogl/renderer/meshdata.py

Type Checking

PicoGL uses mypy for type checking:

Configuration: .. code-block:: ini

[mypy] python_version = 3.7 warn_return_any = True warn_unused_configs = True disallow_untyped_defs = True disallow_incomplete_defs = True check_untyped_defs = True disallow_untyped_decorators = True no_implicit_optional = True warn_redundant_casts = True warn_unused_ignores = True warn_no_return = True warn_unreachable = True strict_equality = True

Usage: .. code-block:: bash

Type check all Python files mypy picogl/

Type check specific file mypy picogl/renderer/meshdata.py

6.11.8 Documentation

Sphinx Configuration

PicoGL uses Sphinx for documentation:

conf.py: .. code-block:: python

import os import sys

sys.path.insert(0, os.path.abspath('..'))

project = 'PicoGL' copyright = '2025, PicoGL Contributors' author = 'PicoGL Contributors' release = '1.0.0'

extensions = [
'sphinx.ext.autodoc', 'sphinx.ext.viewcode', 'sphinx.ext.napoleon', 'sphinx.ext.intersphinx', 'sphinx.ext.todo', 'sphinx.ext.coverage', 'sphinx.ext.mathjax', 'sphinx.ext.ifconfig', 'sphinx.ext.githubpages',

]

html_theme = 'sphinx_rtd_theme' latex_engine = 'xelatex'

Building Documentation: .. code-block:: bash

Build HTML documentation sphinx-build -b html doc/ doc/_build/html

Build PDF documentation sphinx-build -b latex doc/ doc/_build/latex cd doc/_build/latex && make

Docstring Style

PicoGL uses Google-style docstrings:

Example: .. code-block:: python

def create_mesh_data(vertices, colors, normals=None):
"""Create mesh data from vertex information.

Args:
vertices: Array of vertex positions (N, 3) colors: Array of vertex colors (N, 3) normals: Optional array of vertex normals (N, 3)

Returns:
MeshData: Mesh data object

Raises:
ValueError: If vertices or colors are invalid TypeError: If input types are incorrect

""" pass

6.11.9 Contributing

Getting Started

1. **Fork the repository** on GitHub
2. **Clone your fork** locally

3. **Create a feature branch**
4. **Make your changes**
5. **Add tests** for new functionality
6. **Run the test suite**
7. **Submit a pull request**

Example: .. code-block:: bash

Fork and clone git clone https://github.com/your-username/picogl.git cd picogl

Create feature branch git checkout -b feature/new-feature

Make changes # … edit files …

Run tests python -m unittest discover tests

Commit changes git add . git commit -m "Add new feature"

Push to fork git push origin feature/new-feature

Create pull request on GitHub

Code Review Process

Pull Request Requirements: * All tests must pass * Code must be formatted with Black * Code must pass flake8 linting * Code must pass mypy type checking * New functionality must have tests * Documentation must be updated

Review Checklist: * [ ] Code follows project style guidelines * [ ] Tests cover new functionality * [ ] Documentation is updated * [ ] No breaking changes without discussion * [ ] Performance impact is considered * [ ] Cross-platform compatibility is maintained

6.11.10 Release Process

Version Numbering

PicoGL uses semantic versioning (MAJOR.MINOR.PATCH):

- **MAJOR**: Breaking changes
- **MINOR**: New features (backward compatible)
- **PATCH**: Bug fixes (backward compatible)

Examples: * 1.0.0: Initial release * 1.1.0: New features added * 1.1.1: Bug fixes * 2.0.0: Breaking changes

Release Steps

1. **Update version numbers** in all relevant files
2. **Update CHANGELOG.md** with new features and fixes
3. **Run full test suite** to ensure everything works
4. **Build documentation** and verify it's correct
5. **Create release tag** on GitHub
6. **Build and upload** to PyPI
7. **Announce release** to community

Example: .. code-block:: bash

Update version sed -i 's/version = "1.0.0"/version = "1.1.0"/' pyproject.toml

Update changelog # … edit CHANGELOG.md …

Run tests python -m unittest discover tests

Build documentation sphinx-build -b html doc/ doc/_build/html

Create tag git tag -a v1.1.0 -m "Release version 1.1.0" git push origin v1.1.0

Build and upload python -m build python -m twine upload dist/*

6.11.11 Performance Considerations

Rendering Performance

Modern OpenGL: * Use VAOs for efficient rendering * Minimize state changes * Use instanced rendering for repeated objects * Implement frustum culling

Legacy OpenGL: * Use VBOs when available * Minimize immediate mode calls * Batch similar operations * Use display lists for static geometry

Example: .. code-block:: python

Efficient rendering def render_scene(self):

Set up state once glEnable(GL_DEPTH_TEST) glEnable(GL_CULL_FACE)

Render all objects with same state for obj in self.objects:

if obj.visible:
obj.render()

Clean up state glDisable(GL_CULL_FACE) glDisable(GL_DEPTH_TEST)

Memory Management

Buffer Management: * Upload data once and reuse * Delete unused buffers * Use appropriate buffer types * Monitor memory usage

Example: .. code-block:: python

class MeshManager:

def __init__(self):
self.meshes = { } self.buffers = { }

def load_mesh(self, name, data):

if name not in self.meshes:
Create new mesh mesh = self.create_mesh(data) self.meshes[name] = mesh self.buffers[name] = mesh.get_buffers()

return self.meshes[name]

def cleanup(self):

for buffers in self.buffers.values():

for buffer in buffers:
buffer.delete()

self.buffers.clear() self.meshes.clear()

6.11.12 Debugging

OpenGL Debugging

Error Checking: .. code-block:: python

def check_gl_error():
error = glGetError() if error != GL_NO_ERROR:

print(f"OpenGL error: {error}") return False return True

Use after OpenGL calls glDrawElements(GL_TRIANGLES, count, GL_UNSIGNED_INT, None) check_gl_error()

Debug Output: .. code-block:: python

def debug_print_gl_info():
print(f"OpenGL version: {glGetString(GL_VERSION)}") print(f"Vendor: {glGetString(GL_VENDOR)}") print(f"Renderer: {glGetString(GL_RENDERER)}") print(f"Extensions: {glGetString(GL_EXTENSIONS)}")

Performance Profiling: .. code-block:: python

import time

def profile_render():
start_time = time.time()

Render scene self.render_scene()

end_time = time.time() frame_time = end_time - start_time fps = 1.0 / frame_time

print(f"Frame time: {frame_time:.3f}s, FPS: {fps:.1f}")

6.11.13 Common Issues

OpenGL Context Issues: * Ensure OpenGL context is created before use * Check for context loss * Handle context recreation

Memory Issues: * Monitor buffer usage * Clean up unused resources * Use appropriate data types

Performance Issues: * Profile rendering code * Optimize draw calls * Use appropriate rendering techniques

Cross-platform Issues: * Test on multiple platforms * Handle platform-specific differences * Use fallback implementations

For more information, see the *Troubleshooting* guide.

6.12 Testing

PicoGL includes a comprehensive testing framework to ensure reliability and compatibility across different platforms and OpenGL versions.

6.12.1 Test Structure

The test suite is organized as follows:

```
tests/
├── test_vertex_array_object.py      # VAO tests
├── test_vertex_buffer_group.py      # Legacy VBO tests
├── test_meshdata.py                 # MeshData tests
├── test_glmesh.py                   # Modern GLMesh tests
├── test_legacy_glmesh.py            # Legacy GLMesh tests
├── test_object_renderer.py          # ObjectRenderer tests
└── test_texture_renderer.py         # TextureRenderer tests
```

6.12.2 Running Tests

Basic Test Execution

Run all tests: .. code-block:: bash

python -m unittest discover tests

Run specific test file: .. code-block:: bash

python -m unittest tests.test_vertex_array_object

Run specific test method: .. code-block:: bash

python -m unittest tests.test_vertex_array_object.TestVertexArrayObject.test_initialization

Run with verbose output: .. code-block:: bash

python -m unittest discover tests -v

Using pytest

Install pytest: .. code-block:: bash

pip install pytest pytest-cov

Run tests with pytest: .. code-block:: bash

pytest tests/

Run with coverage: .. code-block:: bash

pytest tests/ –cov=picogl

Run specific test: .. code-block:: bash

pytest tests/test_vertex_array_object.py::TestVertexArrayObject::test_initialization

6.12.3 Test Categories

Unit Tests

Unit tests verify individual components in isolation:

VertexArrayObject Tests: .. code-block:: python

class TestVertexArrayObject(unittest.TestCase):

def test_initialization(self):
vao = VertexArrayObject() self.assertIsNotNone(vao)

def test_add_vbo(self):
vao = VertexArrayObject() vao.add_vbo(index=0, data=vertices, size=3) self.assertEqual(len(vao.vbos), 1)

def test_add_ebo(self):
vao = VertexArrayObject() vao.add_ebo(data=indices) self.assertIsNotNone(vao.ebo)

MeshData Tests: .. code-block:: python

class TestMeshData(unittest.TestCase):

def test_from_raw(self):
data = MeshData.from_raw(vertices=vertices, colors=colors) self.assertIsNotNone(data) self.assertEqual(data.vertex_count, len(vertices) // 3)

def test_draw(self):
data = MeshData.from_raw(vertices=vertices, colors=colors) data.draw() # Should not raise exception

Integration Tests

Integration tests verify component interactions:

Renderer Integration: .. code-block:: python

class TestRendererIntegration(unittest.TestCase):

def test_object_renderer_initialization(self):
context = GLContext() data = MeshData.from_raw(vertices=vertices, colors=colors) renderer = ObjectRenderer(context=context, data=data) renderer.initialize() self.assertTrue(renderer.initialized)

def test_texture_renderer_initialization(self):
context = GLContext() data = MeshData.from_raw(vertices=vertices, uvs=uvs) renderer = TextureRenderer(context=context, data=data) renderer.initialize_textures() self.assertIsNotNone(renderer.texture)

Performance Tests

Performance tests verify rendering performance:

Rendering Performance: .. code-block:: python

class TestRenderingPerformance(unittest.TestCase):

def test_render_performance(self):
start_time = time.time() for _ in range(1000):

renderer.render()

end_time = time.time() frame_time = (end_time - start_time) / 1000 self.assertLess(frame_time, 0.016) # 60 FPS

def test_memory_usage(self):
initial_memory = psutil.Process().memory_info().rss # Create and use renderer renderer = create_renderer() renderer.render() final_memory = psutil.Process().memory_info().rss memory_increase = final_memory - initial_memory self.assertLess(memory_increase, 100 * 1024 * 1024) # 100MB

Compatibility Tests

Compatibility tests verify cross-platform functionality:

OpenGL Version Compatibility: .. code-block:: python

class TestOpenGLCompatibility(unittest.TestCase):

def test_modern_opengl(self):

if has_modern_opengl():
vao = VertexArrayObject() self.assertIsNotNone(vao)

def test_legacy_opengl(self):

if has_legacy_opengl():
vbg = VertexBufferGroup() self.assertIsNotNone(vbg)

def test_immediate_mode(self):
Test immediate mode fallback pass

6.12.4 Test Mocking

OpenGL Mocking

Since OpenGL requires a graphics context, tests use mocking:

Global OpenGL Mocking: .. code-block:: python

class TestVertexArrayObject(unittest.TestCase):

def setUp(self):
Mock all OpenGL functions self.gl_patches = [

patch('picogl.backend.modern.core.vertex.array.object.glGenVertexArrays'),
patch('picogl.backend.modern.core.vertex.array.object.glBindVertexArray'),
patch('picogl.backend.modern.core.vertex.array.object.glDeleteVertexArrays'), #
... more patches

] for patch_obj in self.gl_patches:

patch_obj.start()

def tearDown(self):

for patch_obj in self.gl_patches:
patch_obj.stop()

Specific Function Mocking: .. code-block:: python

def test_error_handling(self):

with patch('picogl.backend.modern.core.vertex.array.object.glGenVertexArrays') as mock_gen:
mock_gen.side_effect = OpenGLError("Context not available") with self.assertRaises(OpenGLError):

VertexArrayObject()

Dependency Mocking

Mock external dependencies:

NumPy Mocking: .. code-block:: python

@patch('numpy.array') def test_array_creation(self, mock_array):

mock_array.return_value = np.array([1, 2, 3]) result = create_vertex_array() mock_array.assert_called_once()

PIL Mocking: .. code-block:: python

@patch('PIL.Image.open') def test_texture_loading(self, mock_open):

mock_image = MagicMock() mock_open.return_value = mock_image texture = load_texture("test.png") mock_open.assert_called_once_with("test.png")

6.12.5 Test Data

Test Fixtures

Create reusable test data:

Vertex Data: .. code-block:: python

class TestData:
@staticmethod def create_triangle_vertices():

return np.array([
[0, 0, 0], [1, 0, 0], [0, 1, 0]

], dtype=np.float32)

@staticmethod def create_triangle_colors():

return np.array([
[1, 0, 0], [0, 1, 0], [0, 0, 1]

], dtype=np.float32)

@staticmethod def create_triangle_faces():

return np.array([
[0, 1, 2]

], dtype=np.uint32)

Mock Objects: .. code-block:: python

class MockGLContext:

def __init__(self):
self.vaos = {} self.shader = MagicMock() self.mvp_matrix = np.identity(4, dtype=np.float32)

def create_shader_program(self, vertex_file, fragment_file):
self.shader = MagicMock() return self.shader

Test Utilities

Create helper functions for testing:

OpenGL Context Testing: .. code-block:: python

def create_mock_opengl_context():
""""Create a mock OpenGL context for testing."""" with patch('OpenGL.GL.glGetString') as mock_get_string:

mock_get_string.return_value = b"OpenGL 3.3.0" return mock_get_string

Error Testing: .. code-block:: python

def test_with_gl_error(error_code, expected_exception):
""""Test function with specific OpenGL error."""" with patch('OpenGL.GL.glGetError') as mock_get_error:

mock_get_error.return_value = error_code with pytest.raises(expected_exception):

function_that_uses_opengl()

6.12.6 Continuous Integration

GitHub Actions

Test Workflow: .. code-block:: yaml

name: Tests on: [push, pull_request]

jobs:

test:
runs-on: ${{ matrix.os }} strategy:

matrix:
os: [ubuntu-latest, windows-latest, macos-latest] python-version: [3.7, 3.8, 3.9, '3.10', '3.11']

steps: - uses: actions/checkout@v2 - name: Set up Python

uses: actions/setup-python@v2 with:

python-version: ${{ matrix.python-version }}

- name: Install dependencies run: |

  pip install -e .[dev]

- name: Run tests run: |

  python -m unittest discover tests

- name: Run coverage run: |

pytest tests/ –cov=picogl –cov-report=xml

- name: Upload coverage uses: codecov/codecov-action@v1

Build Workflow: .. code-block:: yaml

name: Build on: [push, release]

jobs:

build:
runs-on: ${{ matrix.os }} strategy:

matrix:
os: [ubuntu-latest, windows-latest, macos-latest]

steps: - uses: actions/checkout@v2 - name: Set up Python

uses: actions/setup-python@v2 with:

python-version: '3.9'

- name: Install dependencies run: |

  pip install build twine

- name: Build package run: |

  python -m build

- name: Upload artifacts uses: actions/upload-artifact@v2 with:

  name: dist path: dist/

6.12.7 Test Coverage

Coverage Requirements

Minimum Coverage: * Overall: 80% * Critical modules: 90% * New code: 95%

Coverage Exclusions: * Test files * Example files * Platform-specific code * Fallback implementations

Coverage Report: .. code-block:: bash

pytest tests/ –cov=picogl –cov-report=html –cov-report=term

Coverage Configuration: .. code-block:: ini

[coverage:run] source = picogl omit =

/tests/ /examples/ /__pycache__/ /legacy/

[coverage:report] exclude_lines =

pragma: no cover def __repr__ raise AssertionError raise NotImplementedError

6.12.8 Code Quality

Linting

flake8 Configuration: .. code-block:: ini

[flake8] max-line-length = 88 extend-ignore = E203, W503 exclude =

.git, __pycache__, .venv, _build, dist

Run Linting: .. code-block:: bash

flake8 picogl/ flake8 tests/

Type Checking

mypy Configuration: .. code-block:: ini

[mypy] python_version = 3.7 warn_return_any = True warn_unused_configs = True disallow_untyped_defs = True disallow_incomplete_defs = True check_untyped_defs = True disallow_untyped_decorators = True no_implicit_optional = True warn_redundant_casts = True warn_unused_ignores = True warn_no_return = True warn_unreachable = True strict_equality = True

Run Type Checking: .. code-block:: bash

mypy picogl/ mypy tests/

Formatting

black Configuration: .. code-block:: toml

[tool.black] line-length = 88 target-version = ['py37'] include = '.pyi?$' extend-exclude = '" /(

.git

.mypy_cache
.tox
.venv
_build
buck-out
build
dist

Run Formatting: .. code-block:: bash

black picogl/ black tests/

6.12.9 Test Best Practices

Writing Tests

Test Naming: .. code-block:: python

def test_initialization_with_valid_data(self):
"""Test initialization with valid input data.""" pass

def test_initialization_raises_error_with_invalid_data(self):
"""Test initialization raises error with invalid data.""" pass

Test Structure: .. code-block:: python

def test_functionality(self):
Arrange input_data = create_test_data() expected_result = create_expected_result()

Act actual_result = function_under_test(input_data)

Assert self.assertEqual(actual_result, expected_result)

Test Documentation: .. code-block:: python

def test_complex_functionality(self):
"""Test complex functionality with multiple inputs.

This test verifies that the function handles: - Valid input data - Edge cases - Error conditions

Expected behavior: - Returns correct result for valid input - Handles edge cases gracefully - Raises appropriate errors for invalid input """ pass

Test Maintenance

Keep Tests Simple: * One assertion per test * Clear test names * Minimal setup

Avoid Test Dependencies: * Tests should be independent * No shared state * Clean up after tests

Regular Test Updates: * Update tests when code changes * Remove obsolete tests * Add tests for new features

Test Performance: * Keep tests fast * Use mocking to avoid slow operations * Parallel test execution when possible

Debugging Tests

Verbose Output: .. code-block:: bash

python -m unittest discover tests -v

Debug Specific Test: .. code-block:: python

import pdb

def test_debugging(self):
pdb.set_trace() # Set breakpoint # Test code here

Test Isolation: .. code-block:: python

def test_isolated(self):
Use fresh instances context = GLContext() data = MeshData.from_raw(vertices=vertices, colors=colors) # Test without side effects

For more information about testing, see the *Development* guide.

6.13 Contributing

Thank you for your interest in contributing to PicoGL! This guide will help you get started with contributing to the project.

6.13.1 Getting Started

Prerequisites

Before contributing, ensure you have:

- Python 3.7 or higher
- Git installed
- A GitHub account
- Basic knowledge of OpenGL and Python

Fork and Clone

1. **Fork the repository** on GitHub
2. **Clone your fork** locally:

```
git clone https://github.com/your-username/picogl.git
cd picogl
```

3. **Add upstream remote**:

```
git remote add upstream https://github.com/original-org/picogl.git
```

4. **Create a feature branch**:

```
git checkout -b feature/your-feature-name
```

6.13.2 Development Setup

Install Dependencies

Install in development mode: .. code-block:: bash

pip install -e .[dev]

Install additional dependencies: .. code-block:: bash

pip install -e .[dev,qt]

Verify installation: .. code-block:: bash

python -c "import picogl; print('PicoGL installed successfully')"

Set Up Pre-commit Hooks

Install pre-commit: .. code-block:: bash

pip install pre-commit

Install hooks: .. code-block:: bash

pre-commit install

Run hooks manually: .. code-block:: bash

pre-commit run –all-files

6.13.3 Development Workflow

Making Changes

1. **Create a feature branch**: .. code-block:: bash

   git checkout -b feature/your-feature-name

2. **Make your changes**: - Write code following the style guidelines - Add tests for new functionality - Update documentation as needed

3. **Test your changes**: .. code-block:: bash

   python -m unittest discover tests pytest tests/ –cov=picogl

4. **Format and lint**: .. code-block:: bash

   black picogl/ tests/ flake8 picogl/ tests/ mypy picogl/ tests/

5. **Commit your changes**: .. code-block:: bash

   git add . git commit -m "Add feature: brief description"

6. **Push to your fork**: .. code-block:: bash

   git push origin feature/your-feature-name

7. **Create a pull request** on GitHub

6.13.4 Code Style

Formatting

PicoGL uses Black for code formatting:

Configuration: .. code-block:: toml

[tool.black] line-length = 88 target-version = ['py37'] include = '.pyi?$'

Usage: .. code-block:: bash

Format all Python files black picogl/ tests/

Check formatting black –check picogl/ tests/

Linting

PicoGL uses flake8 for linting:

Configuration: .. code-block:: ini

[flake8] max-line-length = 88 extend-ignore = E203, W503 exclude =

.git, __pycache__, .venv, _build, dist

Usage: .. code-block:: bash

Lint all Python files flake8 picogl/ tests/

Type Checking

PicoGL uses mypy for type checking:

Configuration: .. code-block:: ini

[mypy] python_version = 3.7 warn_return_any = True warn_unused_configs = True disallow_untyped_defs = True disallow_incomplete_defs = True check_untyped_defs = True disallow_untyped_decorators = True no_implicit_optional = True warn_redundant_casts = True warn_unused_ignores = True warn_no_return = True warn_unreachable = True strict_equality = True

Usage: .. code-block:: bash

Type check all Python files mypy picogl/ tests/

Documentation

PicoGL uses Google-style docstrings:

Example: .. code-block:: python

def create_mesh_data(vertices, colors, normals=None):
"""Create mesh data from vertex information.

Args:
vertices: Array of vertex positions (N, 3) colors: Array of vertex colors (N, 3) normals: Optional array of vertex normals (N, 3)

Returns:
MeshData: Mesh data object

Raises:
ValueError: If vertices or colors are invalid TypeError: If input types are incorrect

""" pass

6.13.5 Testing

Writing Tests

Test Structure: .. code-block:: python

class TestNewFeature(unittest.TestCase):

def setUp(self):
Set up test data pass

def tearDown(self):
Clean up test data pass

def test_basic_functionality(self):
“””Test basic functionality.””” # Arrange input_data = create_test_data() expected_result = create_expected_result()

Act actual_result = function_under_test(input_data)

Assert self.assertEqual(actual_result, expected_result)

def test_edge_cases(self):
“””Test edge cases.””” # Test edge cases pass

def test_error_conditions(self):
“””Test error conditions.””” # Test error conditions pass

Test Naming: * Use descriptive names * Include what is being tested * Include expected outcome

Test Documentation: * Document complex tests * Explain test purpose * Include expected behavior

Running Tests

Run all tests: .. code-block:: bash

python -m unittest discover tests

Run specific test: .. code-block:: bash

python -m unittest tests.test_new_feature

Run with coverage: .. code-block:: bash

pytest tests/ –cov=picogl

Run with verbose output: .. code-block:: bash

python -m unittest discover tests -v

Test Requirements

Coverage Requirements: * Overall: 80% * Critical modules: 90% * New code: 95%

Test Categories: * Unit tests for individual functions * Integration tests for component interactions * Performance tests for rendering * Compatibility tests for different platforms

Test Mocking: * Mock OpenGL functions * Mock external dependencies * Use test fixtures for common data

6.13.6 Documentation

Updating Documentation

API Documentation: * Update docstrings for new functions * Add examples for new features * Update type hints

User Documentation: * Update installation guide * Add new examples * Update troubleshooting guide

Developer Documentation: * Update development guide * Add architecture notes * Update contributing guide

Building Documentation

Build HTML documentation: .. code-block:: bash

sphinx-build -b html doc/ doc/_build/html

Build PDF documentation: .. code-block:: bash

sphinx-build -b latex doc/ doc/_build/latex cd doc/_build/latex && make

View documentation: .. code-block:: bash

open doc/_build/html/index.html

6.13.7 Pull Request Process

Creating Pull Requests

1. **Ensure your branch is up to date**: .. code-block:: bash

   git fetch upstream git rebase upstream/main

2. **Push your changes**: .. code-block:: bash

   git push origin feature/your-feature-name

3. **Create pull request** on GitHub: - Use descriptive title - Provide detailed description - Link to related issues - Include screenshots if applicable

4. **Wait for review**: - Address review comments - Make requested changes - Update tests if needed

Pull Request Template

Title: Brief description of changes

Description: * What changes were made * Why changes were made * How changes were tested * Any breaking changes

Checklist: * [ ] Code follows style guidelines * [ ] Tests cover new functionality * [ ] Documentation is updated * [ ] No breaking changes * [ ] Performance impact considered * [ ] Cross-platform compatibility maintained

Review Process

Review Criteria: * Code quality and style * Test coverage and quality * Documentation completeness * Performance impact * Compatibility considerations

Review Timeline: * Initial review within 1 week * Follow-up reviews as needed * Final approval from maintainers

Addressing Reviews: * Respond to all comments * Make requested changes * Ask questions if unclear * Update tests if needed

6.13.8 Types of Contributions

Bug Fixes

Reporting Bugs: * Use GitHub issues * Provide detailed information * Include steps to reproduce * Attach relevant files

Fixing Bugs: * Create bug fix branch * Write tests for the bug * Fix the issue * Verify fix with tests

Bug Fix Template: .. code-block:: text

> **Bug Description**: Brief description of the bug **Steps to Reproduce**: 1. Step 1 2. Step 2 3. Step 3 **Expected Behavior**: What should happen **Actual Behavior**: What actually happens **Environment**: OS, Python version, etc. **Additional Context**: Any other relevant information

Feature Requests

Requesting Features: * Use GitHub issues * Provide detailed description * Explain use case * Consider implementation complexity

Implementing Features: * Create feature branch * Write comprehensive tests * Update documentation * Consider backward compatibility

Feature Request Template: .. code-block:: text

> **Feature Description**: Brief description of the feature **Use Case**: Why is this feature needed **Proposed Solution**: How should it work **Alternatives**: Other approaches considered **Additional Context**: Any other relevant information

Documentation Improvements

Types of Documentation: * API documentation * User guides * Examples * Tutorials * Troubleshooting guides

Improving Documentation: * Fix typos and errors * Add missing information * Improve clarity * Add examples

Documentation Template: .. code-block:: text

> **Documentation Type**: API/User/Example/etc. **Section**: Which section needs improvement **Current State**: What's wrong or missing **Proposed Changes**: What should be changed **Additional Context**: Any other relevant information

Performance Improvements

Performance Issues: * Identify bottlenecks * Measure current performance * Propose optimizations * Test performance impact

Performance Improvements: * Profile code * Implement optimizations * Add performance tests * Measure improvements

Performance Template: .. code-block:: text

> **Performance Issue**: What is slow **Current Performance**: Measured performance **Proposed Optimization**: How to improve **Expected Improvement**: Expected performance gain **Additional Context**: Any other relevant information

Code Quality Improvements

Code Quality Issues: * Identify code smells * Refactor complex code * Improve readability * Add type hints

Code Quality Improvements: * Refactor code * Add type hints * Improve documentation * Add tests

Code Quality Template: .. code-block:: text

> **Code Quality Issue**: What needs improvement **Current State**: Current code quality **Proposed Changes**: How to improve **Expected Benefits**: What will be improved **Additional Context**: Any other relevant information

6.13.9 Community Guidelines

Code of Conduct

Be Respectful: * Use welcoming and inclusive language * Be respectful of differing viewpoints * Accept constructive criticism gracefully

Be Collaborative: * Focus on what is best for the community * Show empathy towards other community members * Help others learn and grow

Be Professional: * Use appropriate language * Be constructive in feedback * Follow project guidelines

Communication

GitHub Issues: * Use for bug reports and feature requests * Provide detailed information * Be responsive to questions

GitHub Discussions: * Use for general questions * Share ideas and feedback * Help other community members

Pull Requests: * Use for code contributions * Provide detailed descriptions * Be responsive to reviews

Email: * Use for sensitive issues * Contact maintainers directly * Use appropriate subject lines

6.13.10 Getting Help

Documentation: * Check the documentation first * Look for similar issues * Read the troubleshooting guide

Community: * Ask questions on GitHub Discussions * Search existing issues * Join community discussions

Maintainers: * Contact maintainers for urgent issues * Use appropriate communication channels * Be patient with responses

Resources: * OpenGL documentation * Python documentation * PyOpenGL documentation * Platform-specific guides

6.13.11 Recognition

Contributors: * All contributors are recognized * Contributors are listed in README * Significant contributions are highlighted

Types of Contributions: * Code contributions * Documentation improvements * Bug reports * Feature requests * Community help

Recognition Process: * Automatic recognition for contributions * Manual recognition for significant contributions * Regular contributor spotlights

Thank you for contributing to PicoGL! Your contributions help make the project better for everyone.

6.14 Changelog

All notable changes to PicoGL will be documented in this file.

6.14.1 Version 1.0.0 (2025-01-06)

Added * Initial release of PicoGL * Core rendering functionality * Modern OpenGL 3.3+ support * Legacy OpenGL 1.x/2.x support * Comprehensive test suite * Documentation and examples

Features * MeshData class for 3D mesh management * ObjectRenderer for modern OpenGL rendering * TextureRenderer for texture mapping * LegacyGLMesh for compatibility * Multiple UI backends (GLUT, Qt) * Shader management system * Buffer management utilities * Data loading utilities

Examples * Basic cube and teapot examples * Legacy compatibility examples * Molecular visualization examples * Texture mapping examples * Shader examples

Testing * Unit tests for all major components * Integration tests * Performance tests * Compatibility tests * Mock OpenGL functions for testing

Documentation * Comprehensive API documentation * User guides and tutorials * Installation instructions * Troubleshooting guide * Development documentation

Compatibility * Python 3.7+ * Windows, macOS, Linux * OpenGL 1.x, 2.x, 3.3+ * Cross-platform compatibility

Dependencies * NumPy for numerical computing * PyOpenGL for OpenGL bindings * PyGLM for matrix operations * Pillow for image processing * Optional Qt support

Known Issues * Some OpenGL context issues on macOS * Limited shader support on older systems * Performance varies by platform

Future Plans * Additional UI backends * More shader examples * Performance optimizations * Extended platform support

CHAPTER

SEVEN

INDICES AND TABLES

- genindex
- modindex
- search

INDEX

Symbols

A

B

C